

Polynomial regression on Runge's function: least squares, Ridge and Lasso with resampling and gradient descent

FYS-STK3155/4155, Project 1

Pietro Monacelli, Giorgio Spreafico, Elia Valenti, and Georgios Kotrotsios
Department of Physics, University of Oslo, N-0316 Oslo, Norway
(Dated: October 5, 2026)

Fitting a polynomial to noisy data raises the questions that every supervised method must answer: how complex the model should be, how its error is estimated, and how its parameters are computed. We fitted Runge's function, sampled at 100 points with noise of standard deviation 0.1, with polynomials up to degree 15 by ordinary least squares, Ridge and Lasso regression, and compared the closed-form solutions with our own codes for gradient descent, momentum, AdaGrad, RMSprop, Adam and stochastic gradient descent. Ten-fold cross-validation selected degree 11 for least squares, 12 for Ridge and 8 for the Lasso, with errors of 0.020, 0.019 and 0.022 that differ by about one standard error. The bootstrap attributes the failure of high-degree least squares to variance, which the penalties reduced by factors of 6 and 50 at degree 12. Plain gradient descent reached the closed-form solution only up to degree 6; a small Ridge penalty restored convergence at every degree. For this problem the penalties do not lower the best attainable error; they make the result insensitive to the degree and make iterative solvers usable.

I. INTRODUCTION

A polynomial is the simplest flexible model that is still linear in its parameters. Its coefficients follow from one linear-algebra operation, and its flexibility is set by a single integer, the degree. This makes polynomial regression a good laboratory for questions that return in every supervised learning method. How complex should the model be? How do we estimate the error it will make on new data? And how do we compute its parameters when no closed-form solution exists?

The function studied here, $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$, is the standard example of what goes wrong when the degree is too high. Runge showed that polynomial interpolation of this function at equidistant points oscillates more and more strongly near the ends of the interval as the degree grows [1]. With noisy data the same flexibility has a second effect: a high-degree polynomial follows the noise of the particular sample it was fitted to. The general statement is the bias-variance tradeoff [2, 3]: the expected error on new data is the sum of a squared bias, which decreases with the complexity of the model, a variance, which increases with it, and the variance of the noise, which no model can remove.

We compare three linear models. Ordinary least squares (OLS) minimises the squared residuals and nothing else. Ridge regression [4] adds a penalty on the squared size of the parameters, which shrinks them and stabilises the fit when the features are nearly collinear. The Lasso [5] penalises the absolute values instead, which can set parameters exactly to zero but leaves no closed-form solution. To choose the degree and the penalty we need an estimate of the error on unseen data from a single data set. We use two resampling methods: the bootstrap [6], which also separates bias from variance, and K -fold cross-validation [2, 3].

The Lasso has to be solved iteratively, and so do the

models of the later projects of this course. We therefore also replace the closed-form OLS and Ridge solutions by gradient descent and study how the learning rate and the conditioning of the problem control its convergence. We then add momentum [7], the adaptive methods AdaGrad [8], RMSprop [9] and Adam [10], and stochastic gradient descent [11]. Gradients are computed analytically and checked against automatic differentiation [12] with JAX [13].

Our own code covers the OLS and Ridge solvers based on the singular value decomposition, the scaling of the data, the bootstrap estimate of bias and variance, gradient descent with all the update rules above, the subgradient method for the Lasso and stochastic gradient descent. It was developed from the weekly notebooks of the course and the starter code of the project description [14, 15], with the help of a large language model. The text of this report was drafted with the same tool. Both uses are declared in Appendix A. Library routines are used for the train/test split, for K -fold cross-validation, and for the coordinate-descent Lasso that serves as reference solution and in the final model selection (Scikit-Learn [16]); for automatic differentiation (JAX); and for the singular value decomposition (NumPy [17]). The figures were made with Matplotlib [18].

Section II presents the data, the models, the bias-variance decomposition with its derivation, the resampling methods and the optimisers. Section III describes the implementation and the tests against library routines and closed-form solutions. Section IV presents and discusses the results in the order of the project description: OLS, Ridge, the bootstrap, cross-validation, gradient descent and its variants, the Lasso, stochastic gradient descent and the final model selection. Section V gives our conclusions, the limitations of the study and possible extensions. The notebook with all code and outputs is available at <https://github.com/georgkot/FYS-STK4155/tree/main/Project1>.

II. METHODS

A. Data and model

We generate n inputs x_i uniformly on $[-1, 1]$ and targets

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (1)$$

with f Runge's function. The base case is $n = 100$ and $\sigma = 0.1$; we also vary n between 40 and 2000 and σ between 0 and 0.3. The noise is σ times a standard normal number from a NumPy generator with the fixed seed 2026. The same seed gives the same inputs and the same noise pattern for every σ , so comparisons between noise levels isolate the effect of its size. The data are split once at random into 80% training and 20% test data, inside the range of 2/3 to 4/5 recommended in the project description [15].

The model is a polynomial of degree d ,

$$\tilde{y}(x) = \theta_0 + \sum_{j=1}^d \theta_j x^j, \quad (2)$$

which is linear in the intercept θ_0 and in the slope parameters $\boldsymbol{\theta} = (\theta_1, \dots, \theta_d)^T$. It has $p = d+1$ fitted parameters.

B. Scaling, centring and the intercept

The design matrix has the columns $x, x^2, \dots, x^d$ and no column of ones. Each column is standardised: we subtract its mean and divide by its standard deviation. The targets are centred by subtracting their mean $\bar{y}$. Means and standard deviations are computed from the training data only and are then applied unchanged to the test data. For centred columns the least-squares intercept is $\hat{\theta}_0 = \bar{y}_{\text{train}}$ [3], so the prediction is $\tilde{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} + \bar{y}_{\text{train}}$. From here on $\mathbf{X} \in \mathbb{R}^{n \times d}$ denotes the standardised training matrix and $\mathbf{y}$ the centred training targets.

We scale for four reasons. (i) The intercept should not be penalised, because a penalty on θ_0 would make the fit depend on the origin chosen for y . Centring removes the intercept from $\boldsymbol{\theta}$, so Ridge and Lasso penalise the slopes only. (ii) Ridge and Lasso are not invariant under a rescaling of the columns. Their penalties treat all θ_j alike, which is meaningful only if the columns have the same scale; without standardisation the result would depend on the units of x . (iii) The speed of gradient descent is set by the condition number of $\mathbf{X}^T \mathbf{X}$ (Sec. II G), which depends on the relative scale of the columns. (iv) The statistics come from the training data because the test data must play the role of data not yet seen. For OLS these arguments are weaker. Its predictions do not change under a shift or rescaling of the columns as long as an intercept is fitted, so for OLS scaling is a numerical precaution; we check this in Sec. IV A. Standardisation changes the size of the columns, not their shape. It does

not remove the near collinearity of the high powers of x , which is the main numerical difficulty of this project.

C. OLS and Ridge regression through the SVD

OLS minimises $\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2$. Ridge regression minimises

$$C_R(\boldsymbol{\theta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2, \quad \lambda \geq 0, \quad (3)$$

with the solution $\hat{\boldsymbol{\theta}}_R = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$; $\lambda = 0$ gives OLS. This is the convention without a factor $1/n$ in front of the squared error, which is also that of `Ridge(alpha= $\lambda$ )` in Scikit-Learn.

We do not form $\mathbf{X}^T \mathbf{X}$, which would square the condition number. Instead we use the thin singular value decomposition $\mathbf{X} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^T$ with singular values $\sigma_0 \geq \sigma_1 \geq \dots \geq \sigma_{d-1}$. As in the lecture notes, σ_i with an index is a singular value, while σ without an index is the noise level. Then

$$\hat{\boldsymbol{\theta}}_R = \sum_{i=0}^{d-1} \frac{\sigma_i}{\sigma_i^2 + \lambda} (\mathbf{u}_i^T \mathbf{y}) \mathbf{v}_i, \quad (4)$$

and the fitted values are

$$\mathbf{X} \hat{\boldsymbol{\theta}}_R = \sum_{i=0}^{d-1} \frac{\sigma_i^2}{\sigma_i^2 + \lambda} \mathbf{u}_i \mathbf{u}_i^T \mathbf{y}. \quad (5)$$

OLS gives every direction $\mathbf{v}_i$ the coefficient $(\mathbf{u}_i^T \mathbf{y})/\sigma_i$. Ridge multiplies it by the shrinkage factor $\sigma_i^2/(\sigma_i^2 + \lambda) \leq 1$: directions with $\sigma_i^2 \gg \lambda$ are kept, directions with $\sigma_i^2 \ll \lambda$ are removed. The noise contributes a variance σ^2/σ_i^2 to the OLS coefficient along $\mathbf{v}_i$ [3], so the directions that Ridge removes are the ones most sensitive to noise. The sum of the shrinkage factors,

$$\text{df}(\lambda) = \sum_{i=0}^{d-1} \frac{\sigma_i^2}{\sigma_i^2 + \lambda}, \quad (6)$$

is the effective number of parameters of the Ridge fit; it decreases from d at $\lambda = 0$ to zero for $\lambda \rightarrow \infty$. For OLS, singular values below $\epsilon_{\text{mach}} \max(n, d) \sigma_0$ are treated as zero and their directions are dropped. This is the pseudoinverse solution, with the cutoff of the least-squares routine of NumPy.

D. The Lasso

The Lasso replaces the squared penalty by the sum of the absolute values. We use the normalised cost of the lecture notes,

$$C_L(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \bar{\lambda} \|\boldsymbol{\theta}\|_1. \quad (7)$$

The bar marks a penalty that multiplies a cost normalised by $1/n$; its values cannot be compared with those

of λ in Eq. (3). The **Lasso** of Scikit-Learn minimises $\frac{1}{2n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|_2^2 + \alpha\|\boldsymbol{\theta}\|_1$, which is half of Eq. (7) with $\alpha = \bar{\lambda}/2$.

The function $|\theta|$ has no derivative at $\theta = 0$, and Eq. (7) has no closed-form minimiser. The subdifferential of $|\theta|$ at zero is the interval $[-1, 1]$: every number in it is the slope of a line that touches $|\theta|$ at zero and stays below it. Any element of the subdifferential can serve as a subgradient, and we use

$$\mathbf{g}_L(\boldsymbol{\theta}) = \frac{2}{n}\mathbf{X}^T(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) + \bar{\lambda} \text{sign}(\boldsymbol{\theta}) \quad (8)$$

with $\text{sign}(0) = 0$, that is, the element 0 of the subdifferential at zero. At the minimum a coefficient is exactly zero when $|\frac{2}{n}\mathbf{x}_j^T(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})| \leq \bar{\lambda}$ [3], where $\mathbf{x}_j$ is column j of $\mathbf{X}$: the penalty then outweighs what the column could still explain. This is how the Lasso selects features. Coordinate descent with soft thresholding, the algorithm of Scikit-Learn, produces such exact zeros. A subgradient step of fixed length does not, as Sec. IV G shows.

E. Error measures and the bias-variance decomposition

We measure the quality of a fit by the mean squared error and the R^2 score,

$$\text{MSE} = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2, \quad R^2 = 1 - \frac{\sum_i (y_i - \tilde{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad (9)$$

evaluated on the training data or on the test data.

The expected test error can be decomposed as follows. Consider a test input x_i with target $y_i = f_i + \varepsilon_i$, $f_i = f(x_i)$, and a model fitted to a training set $\mathcal{L}$ drawn at random. Its prediction $\tilde{y}_i = \tilde{y}(x_i; \mathcal{L})$ is a random variable, because another training set would give another fit. The expectation $\mathbb{E}$ is taken over two sources of randomness: the training set $\mathcal{L}$, and the noise ε_i of the test target, which is independent of $\mathcal{L}$. The test inputs are held fixed and f_i is not random. We add and subtract the mean prediction $\mathbb{E}[\tilde{y}_i]$,

$$y_i - \tilde{y}_i = \underbrace{f_i - \mathbb{E}[\tilde{y}_i]}_{a_i} + \underbrace{\mathbb{E}[\tilde{y}_i] - \tilde{y}_i}_{b_i} + \varepsilon_i. \quad (10)$$

The square is $a_i^2 + b_i^2 + \varepsilon_i^2$ plus three cross terms whose expectations vanish: $\mathbb{E}[a_i b_i] = a_i \mathbb{E}[b_i] = 0$, because a_i is fixed and $\mathbb{E}[b_i] = 0$ by construction; $\mathbb{E}[a_i \varepsilon_i] = a_i \mathbb{E}[\varepsilon_i] = 0$, because the noise has zero mean; and $\mathbb{E}[b_i \varepsilon_i] = \mathbb{E}[b_i] \mathbb{E}[\varepsilon_i] = 0$, because the test noise is independent of the training set. With $\mathbb{E}[b_i^2] = \text{Var}[\tilde{y}_i]$ and $\mathbb{E}[\varepsilon_i^2] = \sigma^2$, the average over the n_{test} test points is

$$\begin{aligned} \mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] &= \frac{1}{n_{\text{test}}} \sum_i (f_i - \mathbb{E}[\tilde{y}_i])^2 \\ &\quad + \frac{1}{n_{\text{test}}} \sum_i \mathbb{E}[(\tilde{y}_i - \mathbb{E}[\tilde{y}_i])^2] + \sigma^2 \\ &= \text{Bias}^2[\tilde{y}] + \text{Var}[\tilde{y}] + \sigma^2. \end{aligned} \quad (11)$$

The squared bias measures how far the average prediction, over all training sets, is from the true function. It comes from the limits of the model family (a polynomial of low degree cannot produce the narrow peak of f), and more data do not remove it. The variance measures how much the prediction changes when the training set is redrawn: a flexible model follows the noise of the sample it is given. The third term is the variance of the noise in the test target, which no model can predict; it is a lower bound for the expected test error. Raising the degree lowers the bias and raises the variance, so their sum has a minimum at an intermediate complexity.

In practice f is unknown, and the bias is computed with the test data y_i in place of f_i . Since

$$\begin{aligned} \frac{1}{n_{\text{test}}} \sum_i (y_i - \mathbb{E}[\tilde{y}_i])^2 &= \text{Bias}^2 + \frac{1}{n_{\text{test}}} \sum_i \varepsilon_i^2 \\ &\quad + \frac{2}{n_{\text{test}}} \sum_i \varepsilon_i (f_i - \mathbb{E}[\tilde{y}_i]), \end{aligned} \quad (12)$$

where the last term has zero mean and the second has mean σ^2 , the measured bias equals $\text{Bias}^2 + \sigma^2$ on average. It absorbs the noise variance: the decomposition then reads error = measured bias² + variance, without a separate noise term, and the measured bias cannot fall much below σ^2 even for a perfect model.

F. Resampling: bootstrap and cross-validation

Bootstrap. Equation (11) needs an ensemble of training sets, but we have only one. The bootstrap [6] creates B new training sets by drawing n_{train} points with replacement from the training set. Each resample contains on average only $1 - (1 - 1/n_{\text{train}})^{n_{\text{train}}} \approx 63\%$ of the distinct training points [3]. A polynomial is fitted to every resample, scaled with the statistics of that resample, and all B models predict the same test points. With $\tilde{y}_{ib}$ the prediction of model b at test point i and $\bar{y}_i = \frac{1}{B} \sum_b \tilde{y}_{ib}$, the estimates are

$$\begin{aligned} \text{error} &= \frac{1}{n_{\text{test}}} \sum_i \frac{1}{B} \sum_b (y_i - \tilde{y}_{ib})^2, \\ \text{bias}^2 &= \frac{1}{n_{\text{test}}} \sum_i (y_i - \bar{y}_i)^2, \\ \text{variance} &= \frac{1}{n_{\text{test}}} \sum_i \frac{1}{B} \sum_b (\tilde{y}_{ib} - \bar{y}_i)^2. \end{aligned} \quad (13)$$

The bias is the measured one, which contains σ^2 , and the three estimates satisfy error = bias² + variance exactly. We use $B = 100$ and the split of Sec. II A.

Cross-validation. In K -fold cross-validation [2] the data are shuffled and divided into K folds. Each fold is held out once while the model is fitted to the other $K - 1$ folds, and the cross-validated (CV) MSE is the mean of the K held-out errors. Every point is used once for testing and $K - 1$ times for training. We apply it to all

100 points of the base case with $K = 5$ and $K = 10$, using `KFold` with shuffling and `cross_val_score` of Scikit-Learn, with the same folds for every degree and every penalty. As a measure of the uncertainty of a CV estimate we use its standard error, the standard deviation of the K fold errors divided by $\sqrt{K}$.

The model handed to the cross-validation is a pipeline: polynomial features, standardisation, and regression with an unpenalised intercept. The whole pipeline is refitted on the training folds of every split, so the scaler learns its means and standard deviations from the training folds only, and the held-out fold is transformed with these numbers. This matters because the held-out fold must be treated exactly like future data. If the scaler were fitted once on all data, every training fold would contain information about its own held-out fold, and the CV error would be biased towards optimistic values. For scaling alone the effect is small, and for OLS it vanishes. It becomes important for the penalised methods, which are not scale invariant, and for any preprocessing that uses the targets [3].

G. Gradient descent and the learning rate

For gradient descent we use the cost normalised by the number of data points,

$$C(\theta) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\theta\|_2^2 + \bar{\lambda} \|\theta\|_2^2, \quad (14)$$

with gradient and Hessian

$$\nabla C = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y}) + 2\bar{\lambda}\theta, \quad \mathbf{H} = \frac{2}{n} \mathbf{X}^T \mathbf{X} + 2\bar{\lambda}\mathbf{I}, \quad (15)$$

where $\bar{\lambda} = 0$ is OLS. The minimiser of Eq. (14) is $(\mathbf{X}^T \mathbf{X} + n\bar{\lambda}\mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$, the Ridge solution of Eq. (4) with $\lambda = n\bar{\lambda}$. The factor n must be included when gradient descent is compared with the closed form; otherwise the two minimise different costs. With our $n = 80$ training points, $\bar{\lambda} = 10^{-2}$ corresponds to $\lambda = 0.8$.

Gradient descent iterates

$$\theta_{k+1} = \theta_k - \eta \nabla C(\theta_k) \quad (16)$$

with a fixed learning rate η . For a quadratic cost the error $\theta_k - \hat{\theta}$ along the eigenvector of $\mathbf{H}$ with eigenvalue λ_i is multiplied by $1 - \eta\lambda_i$ in every step [3]. (Eigenvalues of the Hessian always carry an index, as in λ_i , $\lambda_{\max}$ and $\lambda_{\min}$; the penalty never does.) The iteration therefore converges if and only if

$$\eta < \frac{2}{\lambda_{\max}}. \quad (17)$$

The fastest fixed rate is $\eta^* = 2/(\lambda_{\max} + \lambda_{\min})$, and at η^* the error shrinks by the factor $(\kappa - 1)/(\kappa + 1)$ per step, where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number of the Hessian. For large κ the number of iterations needed

for a given accuracy is proportional to κ . In terms of the singular values of $\mathbf{X}$ the eigenvalues are $\lambda_i = 2\sigma_i^2/n + 2\bar{\lambda}$. For OLS this gives $\kappa(\mathbf{H}) = \kappa(\mathbf{X})^2$, and a positive $\bar{\lambda}$ lifts the smallest eigenvalue and lowers κ .

All runs start from $\theta_0 = \mathbf{0}$. They stop when $\|\theta_k - \hat{\theta}\|_2 < 10^{-6}$, where $\hat{\theta}$ is the closed-form solution, when the distance exceeds 10^6 (divergence), or after a maximum number of iterations (10^5 for plain gradient descent, 2×10^4 in the comparison of the optimisers).

Automatic differentiation. The gradient in Eq. (15) is also computed by automatic differentiation (AD) with `jax.grad`, applied to the cost written as an ordinary Python function. AD is neither symbolic nor numerical differentiation [3, 12]. Symbolic differentiation manipulates formulas and returns a formula, which can grow much faster than the original expression. Numerical differentiation replaces the derivative by a difference quotient with a step h . It has a truncation error that decreases with h and a rounding error that increases as h decreases, so a central difference reaches a relative accuracy of order 10^{-11} at best in double precision, and a gradient with respect to p parameters costs $2p$ evaluations of the cost. AD applies the exact derivative rule of each elementary operation along the sequence of operations that the program executes, and combines them with the chain rule. It returns the numerical value of the derivative at a point, with no step h and no truncation error, so it is accurate to rounding. In reverse mode, which `jax.grad` uses, the value and the complete gradient of a scalar cost require at most a small constant multiple (at most about four) of the operations of one evaluation of the cost, whatever the number of parameters.

H. Momentum and adaptive learning rates

The other update rules differ in how the step is built from the gradient $\mathbf{g}_t = \nabla C(\theta_{t-1})$ at step $t = 1, 2, \dots$. All operations on vectors are elementwise and all auxiliary vectors start at zero.

$$\begin{aligned} \text{Momentum:} \quad & \mathbf{v}_t = \beta \mathbf{v}_{t-1} + \eta \mathbf{g}_t, \\ & \theta_t = \theta_{t-1} - \mathbf{v}_t. \end{aligned} \quad (18)$$

$$\begin{aligned} \text{AdaGrad:} \quad & \mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t^2, \\ & \theta_t = \theta_{t-1} - \frac{\eta \mathbf{g}_t}{\sqrt{\mathbf{r}_t} + \epsilon}. \end{aligned} \quad (19)$$

$$\begin{aligned} \text{RMSprop:} \quad & \mathbf{r}_t = \rho \mathbf{r}_{t-1} + (1 - \rho) \mathbf{g}_t^2, \\ & \theta_t \text{ as in Eq. (19)}. \end{aligned} \quad (20)$$

$$\begin{aligned} \text{Adam:} \quad & \mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \\ & \mathbf{r}_t = \beta_2 \mathbf{r}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \\ & \theta_t = \theta_{t-1} - \frac{\eta \hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{r}}_t} + \epsilon}, \end{aligned} \quad (21)$$

with the bias-corrected averages $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1 - \beta_1^t)$ and $\hat{\mathbf{r}}_t = \mathbf{r}_t/(1 - \beta_2^t)$. We use the values of the course notebooks: $\beta = 0.9$, $\rho = 0.99$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and

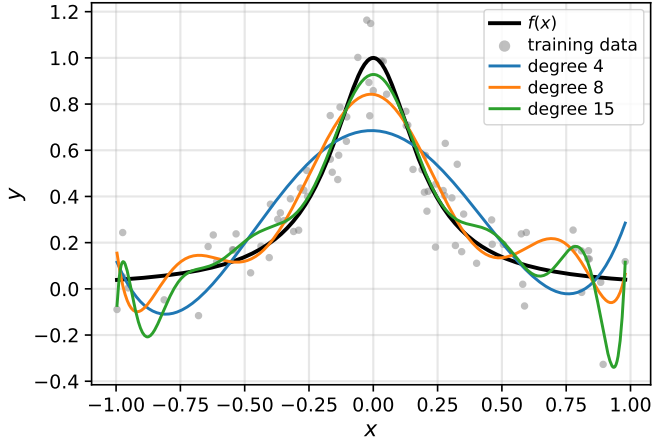


Figure 1. OLS fits of degree 4, 8 and 15 to the base data set ($n = 100$, $\sigma = 0.1$; the 80 training points are shown) and Runge’s function $f(x)$. Degree 4 cannot follow the peak; degree 15 oscillates between the data points and near the ends of the interval.

$\epsilon = 10^{-8}$. For plain gradient descent and momentum, η multiplies the gradient, so the stable range of η is set by the curvature of the cost. For the adaptive methods the gradient is divided by a measure of its own typical size, so η acts as a step length in parameter space [3].

I. Stochastic gradient descent

Stochastic gradient descent (SGD) replaces the full gradient by the gradient over a random minibatch of M of the n training points, which is an unbiased but noisy estimate of the full gradient. One epoch is a pass through all points in n/M minibatches, taken from a new random permutation in every epoch. An epoch costs the same arithmetic as one step with the full gradient, whatever the value of M [3], so we compare results per epoch. The update rules are those of Sec. II H, applied to the minibatch gradient. Besides constant learning rates we use the schedule

$$\eta_t = \frac{t_0}{t + t_1}, \quad (22)$$

where t counts updates: the learning rate starts at t_0/t_1 and decays as $1/t$ after about t_1 updates. Because the SGD iterate fluctuates around the minimum, we measure its accuracy by the excess cost $C(\theta) - C(\hat{\theta})$ over the closed-form minimum, and we summarise the final state by the median over the last 100 epochs.

III. IMPLEMENTATION AND TESTS

All computations are contained in one Jupyter notebook, organised in the parts a) to i) of the project description. Each part defines a few

short functions that later parts reuse: `make_data`, `polynomial_features` and `scale_train_test` for the data and its scaling; `ols_svd` and `ridge_svd` for the closed forms (Listing 1); `bootstrap_bias_variance` for Eq. (13); `gradient_descent`, `optimiser_step`, `optimise_to_accuracy` and `sgd` for the iterative methods. The Scikit-Learn pipelines of Sec. II F are built by three small functions, one for each model. Every cell states which course notebook it was adapted from and what was changed, and the code is commented line by line.

```
def ols_svd(X, y):
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    cutoff = np.finfo(float).eps * max(X.shape) * s[0]
    keep = s > cutoff
    coeffs = (U[:, keep].T @ y) / s[keep]
    return Vt[keep].T @ coeffs

def ridge_svd(X, y, lam):
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    factor = s / (s**2 + lam)
    return Vt.T @ (factor * (U.T @ y))
```

Listing 1. The two closed-form solvers (comments and docstrings removed).

Since the closed-form solutions serve as the reference for all iterative methods, we first tested them against independent library routines, and then tested every further step against a known result. Table I collects these checks. The solvers agree with NumPy and Scikit-Learn to rounding accuracy. The agreement of the Ridge solver with the normal equations degrades from 10^{-15} at $\lambda = 100$ to 10^{-10} at $\lambda = 10^{-4}$. This is expected, since the condition number of $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}$ grows as λ decreases, and it illustrates why we solve through the SVD of $\mathbf{X}$ instead.

All random choices (data, split, bootstrap indices, folds, minibatch order) use the seed 2026. A run of the notebook from top to bottom reproduces every number quoted in this report; only the quantities at rounding level in Table I and the computing times of Sec. IV H change from run to run.

IV. RESULTS AND DISCUSSION

A. OLS: degree, sample size and noise

Figure 1 shows the base data set and three OLS fits. The polynomial of degree 4 is too stiff to follow the peak of f , the fit of degree 8 is closest to f , and the fit of degree 15 follows the peak best but oscillates between the data points and near the ends of the interval. Against the noise-free function on a fine grid, the mean squared distance is 0.0180, 0.0057 and 0.0094 for degrees 4, 8 and 15, and the largest deviation of the degree-15 fit is 0.38 for $|x| > 0.85$ against 0.23 in the interior.

Figure 2 quantifies this. The training MSE decreases with every added power, from 0.094 at $d = 1$ to 0.0124

Table I. Tests of the implementation. Unless stated otherwise the data are those of the base case ($n = 100$, $\sigma = 0.1$) with the 80 standardised training points; “difference” is the largest absolute difference between the two results.

Quantity tested	Compared with	Result
OLS parameters from <code>ols_svd</code> , $d = 15$, all 100 points	<code>numpy.linalg.pinv</code> and <code>lstsq</code>	difference 5×10^{-13} for parameters up to 596
Test predictions of the scaled OLS fit, $d = 15$	unscaled fit with a column of ones; <code>LinearRegression</code> of Scikit-Learn	difference below 10^{-11}
<code>ridge_svd</code> at $\lambda = 0$, $d = 15$	<code>ols_svd</code>	difference below 10^{-12}
<code>ridge_svd</code> at $\lambda = 10^{-4}, 10^{-2}, 1, 100$	normal equations; <code>Ridge(alpha=λ)</code> of Scikit-Learn	relative difference 10^{-10} to 10^{-15}
Fitted values from the shrinkage factors, Eq. (5)	$\mathbf{X}\hat{\boldsymbol{\theta}}_R$	difference below 10^{-14}
Training MSE and $\ \hat{\boldsymbol{\theta}}_R\ _2$ on 131 values of λ	theory: non-decreasing and strictly decreasing in λ	both hold
Bootstrap estimates, Eq. (13)	identity error = bias ² + variance	holds for every degree
Analytical gradients of Eq. (15), OLS and Ridge (unscaled design matrix of the starter code)	<code>jax.grad</code> (64-bit)	difference at most 2×10^{-16}
Gradient descent with the analytical gradient, $d = 5$	the same with the JAX gradient	same iteration counts; parameters differ by less than 10^{-15}
Largest learning rate that converges, grid of 60 values, $d = 5$	bound $2/\lambda_{\max}$, Eq. (17)	0.3594 against 0.3596 (OLS), 0.3581 against 0.3583 (Ridge)
Lasso by subgradient descent, $d = 5$, $\bar{\lambda} = 10^{-2}$	<code>Lasso(alpha=$\bar{\lambda}/2$)</code> of Scikit-Learn	parameters agree to four decimals

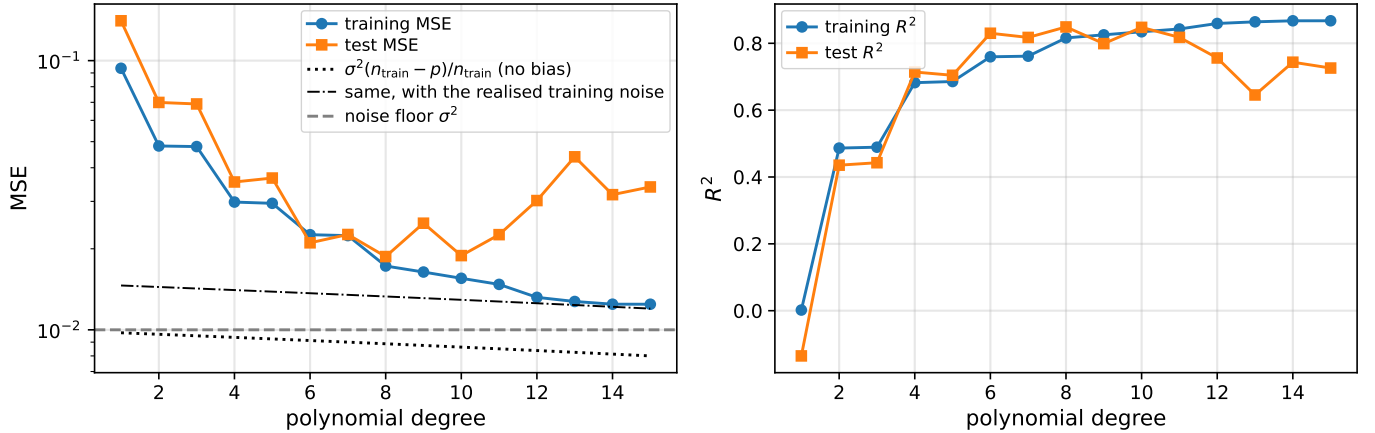


Figure 2. Left: training and test MSE of OLS against the polynomial degree for the base data set ($n = 100$, $\sigma = 0.1$, 80 training and 20 test points). Dotted: expected training MSE $\sigma^2(n_{\text{train}} - p)/n_{\text{train}}$ of a model without bias, with $p = d + 1$; dash-dotted: the same with the noise variance realised in the training points (0.0150); dashed: σ^2 . Right: the corresponding R^2 scores.

at $d = 15$, as it must for nested models. The test MSE is lowest at $d = 8$ (0.0187, $R^2 = 0.849$) and then rises irregularly, to 0.044 at $d = 13$. At $d = 1$ the test R^2 is negative (-0.14): a straight line predicts the test points worse than their own mean does, which is possible for data not used in the fit. Both curves fall in steps, because f is even and an odd power adds almost nothing.

For a model without bias the expected training MSE is $\sigma^2(n_{\text{train}} - p)/n_{\text{train}}$ [3]: the fit absorbs p degrees of freedom of the noise, so the training error lies below σ^2 . The noise realised in our 80 training points has variance

0.0150 rather than 0.0100, so Fig. 2 also shows this curve for the realised value. The training MSE reaches it at $d = 14$ and 15 (0.0124 against 0.0122 and 0.0120). From there on further parameters can only fit noise.

The parameters (Fig. 3) explain why the high degrees fail. The norm $\|\boldsymbol{\theta}\|_2$ grows from 0.86 at $d = 4$ to 11.8 at $d = 8$ and 669 at $d = 15$, while the test error gets worse beyond $d = 8$. The high powers of x are nearly collinear on $[-1, 1]$, so the data hardly determine their individual coefficients, and large coefficients of alternating sign compensate each other. Since f is even, a good fit needs even

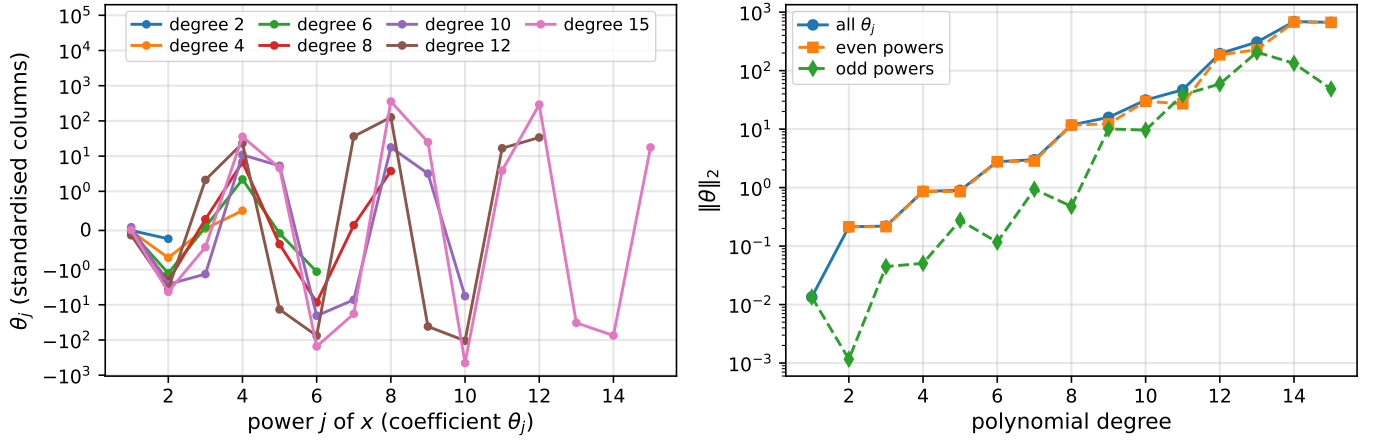


Figure 3. OLS parameters for the base data set. Left: the coefficients θ_j of the standardised columns for selected degrees (the vertical axis is linear between -1 and 1 and logarithmic outside). Right: the norm $\|\theta\|_2$ against the degree, for all coefficients and separately for the coefficients of the even and of the odd powers of x .

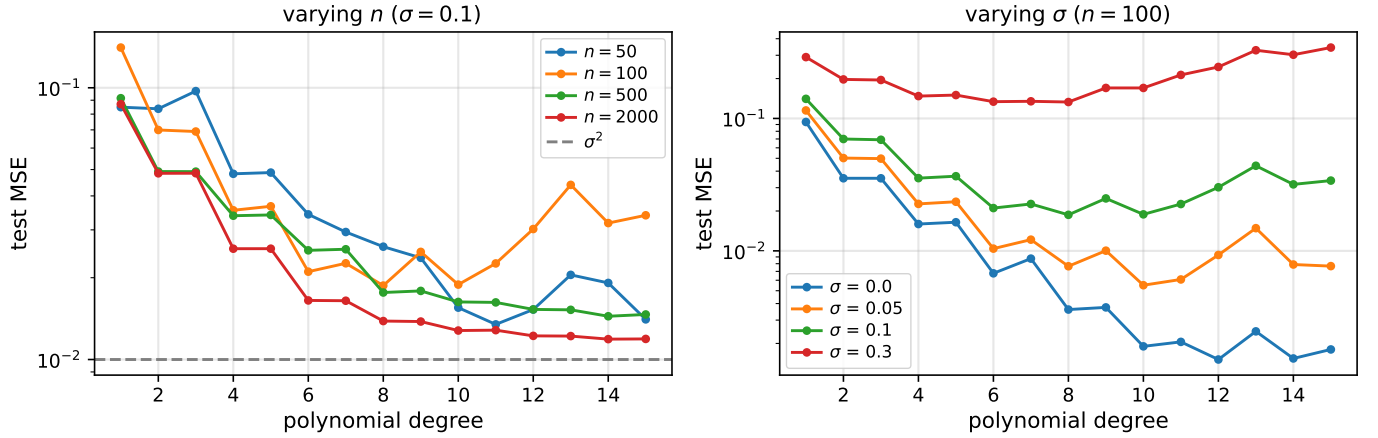


Figure 4. Test MSE of OLS against the degree. Left: for different numbers of data points n at $\sigma = 0.1$. Right: for different noise levels σ at $n = 100$ (same inputs and noise pattern; only the size of the noise changes). Every curve rests on one data set and one 80/20 split.

powers only. Up to $d = 8$ the odd coefficients are indeed negligible (norm 0.48 against 11.7 for the even ones). At $d = 12$ and 15 they reach 59 and 49, a sign that they are no longer determined by the data.

Figure 4 shows the dependence on the number of data points and on the noise. With more data the best degree moves up, from 8 at $n = 100$ to 14 at $n = 500$ and 2000, and the training MSE at $d = 15$ approaches σ^2 (0.0052, 0.0124, 0.0091 and 0.0101 for $n = 50, 100, 500$ and 2000): with more points a flexible model spends its parameters on the function instead of the noise. The curve for $n = 50$ has its minimum at $d = 11$ but rests on only 10 test points. More noise moves the best degree down, 12, 10, 8 and 8 for $\sigma = 0, 0.05, 0.1$ and 0.3 , with a best test R^2 of 0.981, 0.945, 0.849 and 0.505. At $\sigma = 0$ the remaining test MSE (0.0015) is approximation error, and it keeps falling up to $d = 12$. At $n = 100$ the degradation at high degree is therefore not Runge’s phenomenon alone. It comes mainly from the variance that the noise induces,

which is largest near the ends of the interval (Fig. 1).

Effect of the scaling. We checked the arguments of Sec. II B at $d = 15$. The scaled fit, an unscaled fit with a column of ones and the `LinearRegression` of Scikit-Learn give the same test predictions to rounding (Table I), and scaling statistics from all 100 points instead of the training points change nothing for OLS. On $[-1, 1]$ standardisation barely improves the condition number of the design matrix (3.5×10^5 against 2.0×10^5), since all powers are of similar size. Its value shows when the units change. If x is multiplied by 10, the condition number of the unscaled matrix becomes 6×10^{14} , the solver drops 4 of its 16 directions and the test MSE rises from 0.034 to 0.216, whereas the standardised matrix and its fit do not change. Omitting both the column of ones and the centring forces the fit through $\hat{y}(0) = 0$, where $f(0) = 1$, and gives a test MSE of 0.269. Standardisation thus protects against the choice of units and prepares the penalised methods and gradient descent, but it does not cure the

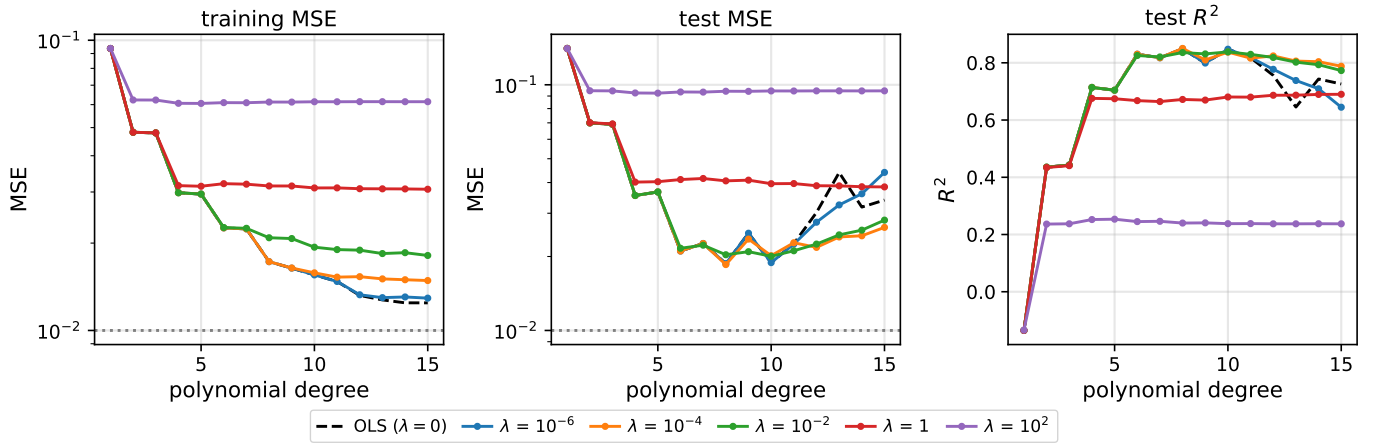


Figure 5. Ridge regression against the degree for five penalties λ [Eq. (3)], with the data and the split of Fig. 2. Left: training MSE; middle: test MSE; right: test R^2 . The dashed black line is OLS ($\lambda = 0$), the dotted line is σ^2 .

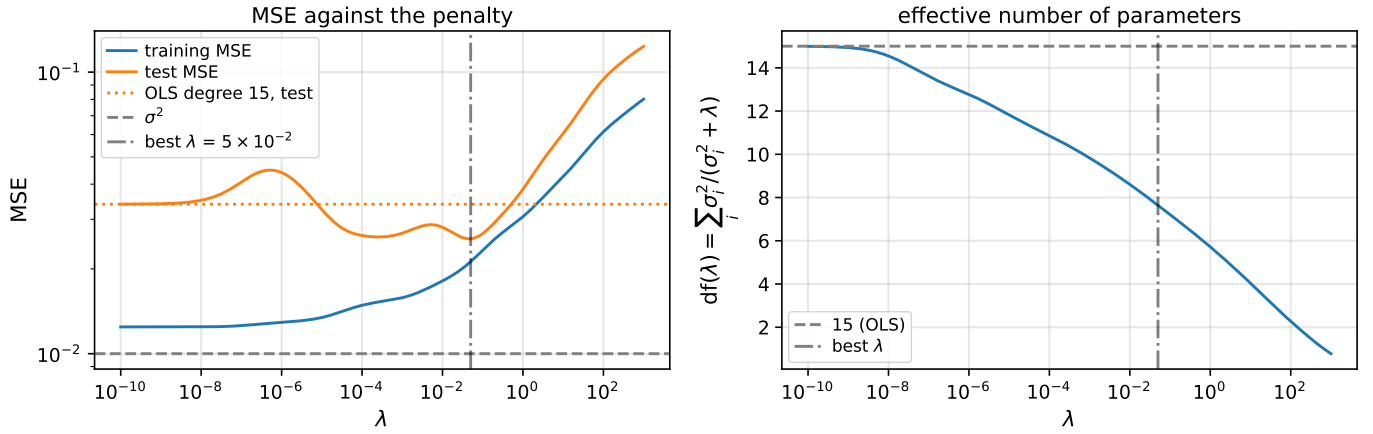


Figure 6. Ridge regression at degree 15 as a function of the penalty. Left: training and test MSE; the dotted line is the test MSE of OLS at degree 15 and the vertical line marks the lowest test MSE ($\lambda = 5 \times 10^{-2}$). Right: effective number of parameters, Eq. (6).

collinearity.

All numbers in this and the next subsection rest on one data set and one split with 20 test points, so single features of the test curves, such as the peak at $d = 13$, should not be over-interpreted.

B. Ridge regression

Figure 5 repeats the analysis for five penalties. Ridge always raises the training error, because OLS is by definition the fit with the smallest training error: at $d = 15$ it rises from 0.0124 to 0.0149, 0.0181, 0.0307 and 0.0614 for $\lambda = 10^{-4}$, 10^{-2} , 1 and 100. Where OLS overfits, a moderate penalty lowers the test error: at $d = 15$ it falls from 0.034 to 0.026 for $\lambda = 10^{-4}$. At low degree there is little variance to remove, and a small penalty changes nothing (0.0185 at $\lambda = 10^{-4}$ and $d = 8$ against 0.0187 for OLS). A large penalty underfits at every degree (test

MSE near 0.09 for $\lambda = 100$), and a small one does not always help on a given split ($\lambda = 10^{-6}$ gives 0.044 at $d = 15$, more than OLS). The degree and λ thus act as two controls of the same model complexity. On a grid of 23 penalties between 10^{-8} and 10^3 and the 15 degrees, the lowest test MSE is 0.0178, at $\lambda = 10^{-3}$ and $d = 8$. This is 5% below the best OLS value, but the point was selected on the test set itself, so the estimate is optimistic and the gain is within the variation between splits.

With the degree fixed at 15, λ is the only control (Fig. 6). For $\lambda < 10^{-8}$ the test MSE equals the OLS value 0.034. It reaches its minimum of 0.0256, 25% lower, at $\lambda = 5 \times 10^{-2}$, and rises steeply for $\lambda \gtrsim 1$, to 0.12 at $\lambda = 10^3$, where the penalty adds more bias than the variance it removes. In between the curve is not monotonic (0.045 near $\lambda = 5 \times 10^{-7}$), which again reflects the single split. The effective number of parameters falls smoothly from 15 to 0.8. At the best λ it is 7.6, close to the 8 slope parameters of the best OLS degree: the two controls set-

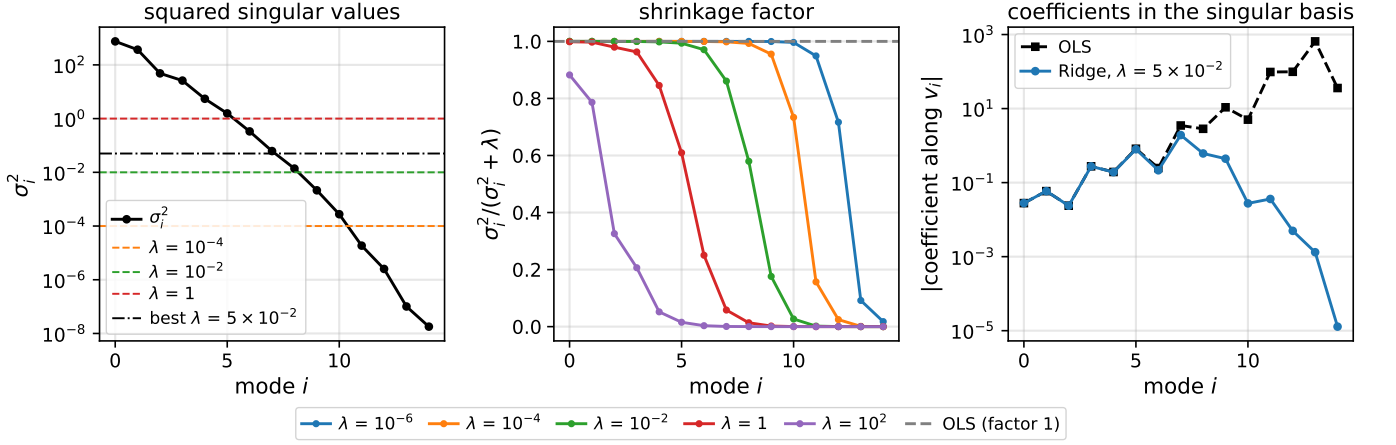


Figure 7. Shrinkage of the singular-value modes of the standardised training matrix at degree 15. Left: squared singular values σ_i^2 , with horizontal lines where $\sigma_i^2 = \lambda$ for four penalties. Middle: shrinkage factor $\sigma_i^2/(\sigma_i^2 + \lambda)$ of Eq. (5). Right: magnitude of the coefficient along each direction $\mathbf{v}_i$ for OLS and for Ridge at $\lambda = 5 \times 10^{-2}$.

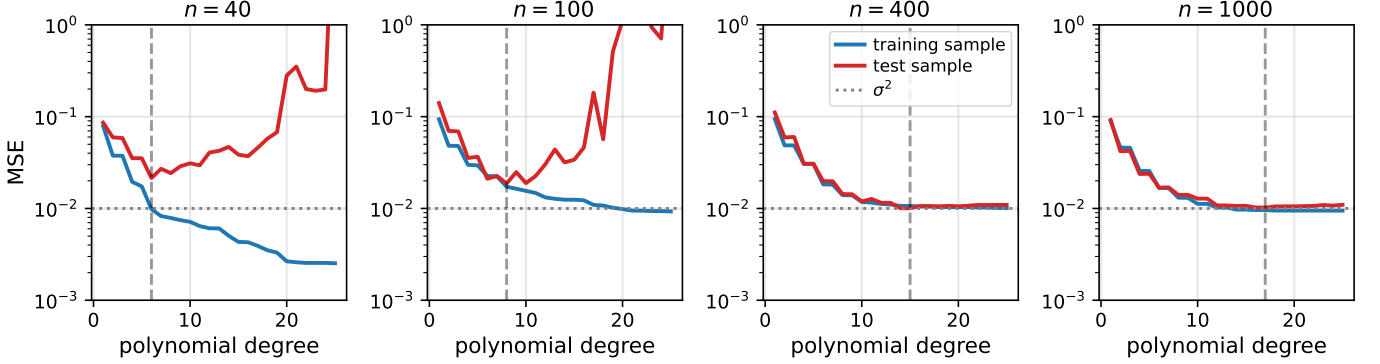


Figure 8. Training and test MSE of OLS against the degree, up to degree 25, for four sample sizes ($\sigma = 0.1$; one data set and one 80/20 split per panel). This is our analogue of Fig. 2.11 of Ref. [2]. The dashed vertical line marks the lowest test MSE and the dotted line σ^2 . Left of the minimum the model has high bias and low variance, right of it low bias and high variance. Test errors above 1 are outside the plotted range.

tle on models of about the same effective size.

Figure 7 shows how this works in the singular basis. The squared singular values of the degree-15 matrix span more than ten orders of magnitude, from 750 to 1.8×10^{-8} , which is a direct measure of the collinearity of the monomials. OLS divides by σ_i , so the weak directions receive large coefficients: about 97, 99 and 654 for the modes 11, 12 and 13. At $\lambda = 5 \times 10^{-2}$ the shrinkage factor is at least 0.87 for the modes 0 to 6, 0.55 for mode 7, 0.22 for mode 8, 0.04 for mode 9 and below 0.01 for the modes 10 to 14, and the three large coefficients become 0.04, 0.005 and 0.001. The transition lies where $\sigma_i^2 \approx \lambda$, and the factors add up to $\text{df} = 7.6$. Because the singular values are spread so widely, a growing λ passes them one after another, which explains the smooth decrease of df in Fig. 6. The norm of the Ridge solution decreases strictly with λ , from 611 at $\lambda = 10^{-8}$ to 19.5, 4.0, 0.63 and 0.08 at $\lambda = 10^{-4}$, 10^{-2} , 1 and 100.

The best penalty depends on the noise and on the amount of data (selected on the test set at $d = 15$, on a

grid of five penalties per decade). It grows with the noise, 6×10^{-8} , 4×10^{-5} , 4×10^{-2} and 2×10^{-1} for $\sigma = 0, 0.05, 0.1$ and 0.3 , since more noise makes more directions not worth keeping. For $n = 500$ and 2000 it is below 10^{-5} and Ridge equals OLS: with enough data the weak directions are well determined. With noise, Ridge at degree 15 remains worse than OLS at its best degree (0.0256 against 0.0187 at $\sigma = 0.1$). On this problem, choosing the degree is at least as effective as penalising. The benefit of Ridge is that it makes the result insensitive to a degree that is too high.

C. Bias-variance tradeoff with the bootstrap

Figure 8 is our version of Fig. 2.11 of Ref. [2], for degrees up to 25 and four sample sizes. In every panel the training MSE decreases monotonically. For $n = 40$ the test MSE is lowest at $d = 6$ (0.0215); beyond that the training MSE falls far below σ^2 (0.0025 at $d = 25$) while

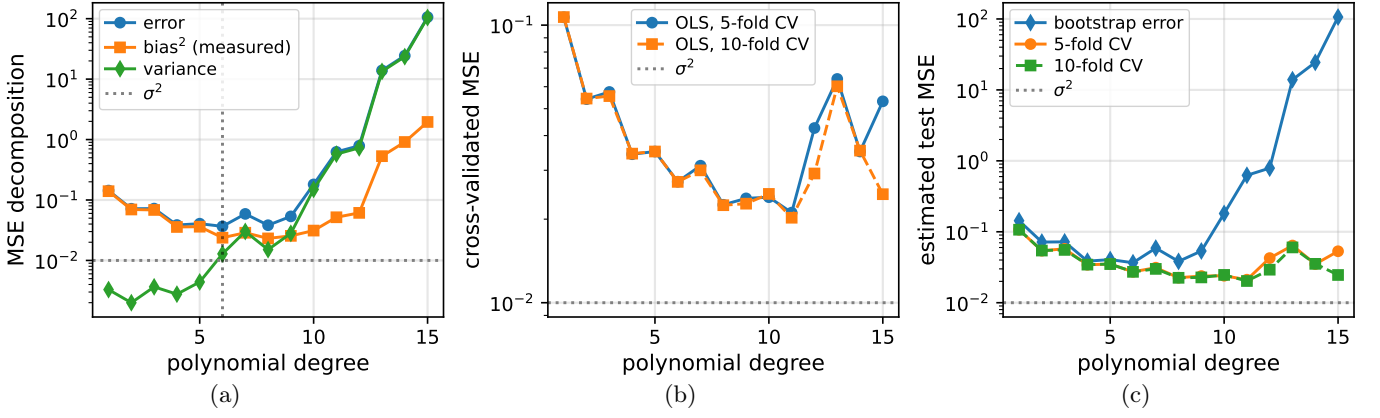


Figure 9. Resampling estimates of the test error of OLS against the degree for the base data set ($n = 100$, $\sigma = 0.1$). (a) Bootstrap estimate of the error and its decomposition, Eq. (13), with $B = 100$; the bias is measured with the test data and contains σ^2 , and the vertical line marks the lowest error, at degree 6. (b) Cross-validated MSE for $K = 5$ and $K = 10$ folds. (c) The two curves of (b) together with the bootstrap error of (a). The dotted horizontal lines are σ^2 .

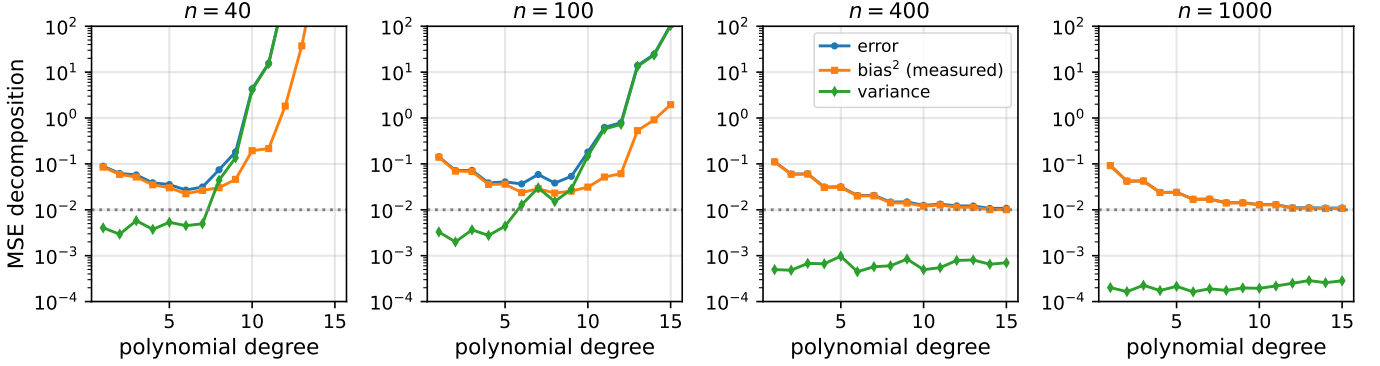


Figure 10. Bootstrap decomposition as in Fig. 9(a) for four sample sizes ($\sigma = 0.1$, $B = 100$). The vertical range is the same in all panels; for $n = 40$ and $n = 100$ the error and the variance leave it at high degree.

the test MSE rises: the model fits the noise. For $n = 100$ the minimum is at $d = 8$ (0.0187), and the test error rises steeply beyond $d = 16$. For $n = 400$ and 1000 the training and the test error stay together near σ^2 up to $d = 25$ (minima at $d = 15$ and 17, both 0.010). With more data, overfitting starts at a higher degree and is weaker. Our curves are less smooth than the schematic figure of Ref. [2], because each rests on a single data set.

The bootstrap separates the two contributions [Fig. 9(a)]. At low degree the variance is small (0.002 to 0.004 for $d \leq 5$): a stiff polynomial hardly changes from one resample to the next, and the error is almost entirely bias, which falls from 0.140 at $d = 1$ to 0.036 at $d = 4$. Around $d = 6$ the variance (0.013) becomes comparable to the measured bias (0.024) and the error has its minimum, 0.0366. The minimum is shallow: the error lies between 0.037 and 0.041 for $d = 4, 5, 6$ and 8. Beyond $d = 9$ the variance grows rapidly, to 0.15 at $d = 10$, 0.73 at $d = 12$ and 104 at $d = 15$, and the error follows it. This is the variance side of the tradeoff: each high-degree model follows the noise of its own resample, and the weak singular directions of Fig. 7 amplify that

noise by σ^2/σ_i^2 .

The measured bias never falls below 0.023, about twice σ^2 , in line with Sec. II E: it contains the noise variance. It also rises at high degree, to 0.91 at $d = 14$ and 1.96 at $d = 15$, although a richer polynomial family should not have a larger bias. We attribute much of this rise to the estimator. The mean prediction $\bar{y}_i$ in Eq. (13) is an average over only $B = 100$ resamples, so it fluctuates with a variance of the order of the bootstrap variance divided by B , and this adds to the measured bias. At $d = 15$ the contribution is $104/100 \approx 1.0$, about half of the measured value. The remainder reflects that a few resamples leave gaps in the data in which a high-degree fit takes extreme values, which also shifts the mean prediction. The bias curve is therefore reliable only where the variance is small, here up to about $d = 9$.

Figure 10 repeats the analysis for other sample sizes. The degree with the lowest error moves up with n : 6, 6, 14 and 14 for $n = 40, 100, 400$ and 1000, with errors of 0.027, 0.037, 0.011 and 0.011. (The errors are not monotonic in n because each rests on its own small test set, 8 points for $n = 40$.) The variance decreases with n

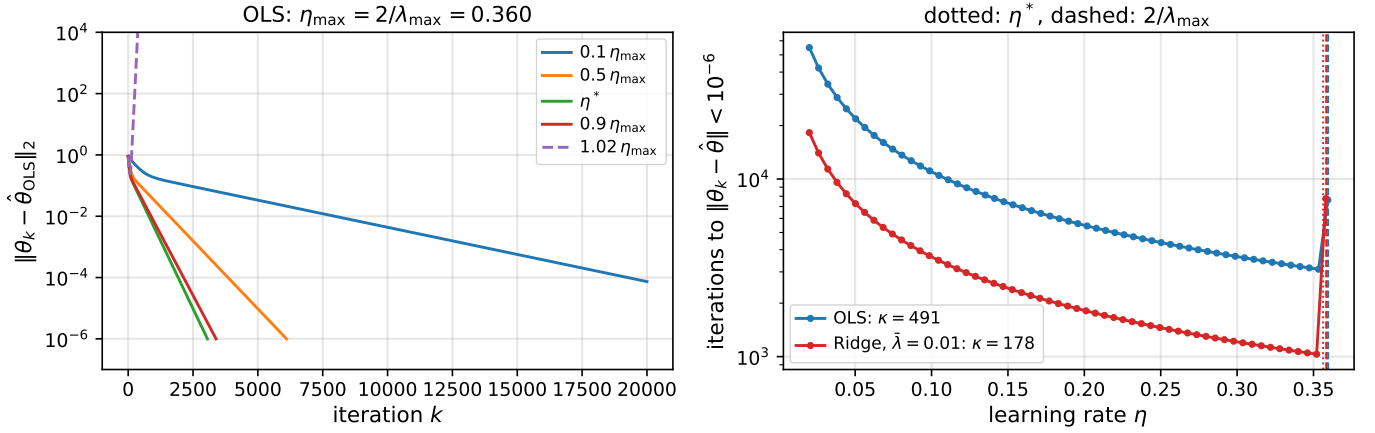


Figure 11. Plain gradient descent at degree 5 (80 standardised training points). Left: distance to the closed-form OLS solution against the iteration, for five learning rates given as fractions of $\eta_{\max} = 2/\lambda_{\max}$. Right: iterations needed to reach $\|\theta_k - \hat{\theta}\|_2 < 10^{-6}$ against the learning rate, for OLS and for Ridge with $\bar{\lambda} = 10^{-2}$; the dotted vertical lines mark η^* and the dashed lines $2/\lambda_{\max}$.

Table II. Degree of the OLS polynomial selected by each estimate of the test error for the base data set ($n = 100$, $\sigma = 0.1$), and the estimated error at that degree.

Estimate	Degree	Error
Single 80/20 split (test MSE)	8	0.0187
Bootstrap, $B = 100$	6	0.0366
5-fold cross-validation	11	0.0211
10-fold cross-validation	11	0.0202

at every degree; at $d = 15$ it is 1.4×10^8 , 104, 7.0×10^{-4} and 2.8×10^{-4} . More data postpone the growth of the variance and make a richer model affordable, while the bias at low degree is the same for all n , since it is a property of the model family. For $n \geq 400$ the measured bias levels off at σ^2 : once the true bias is small, what remains is the noise that it contains.

D. Cross-validation

Figure 9(b) shows the cross-validated MSE of OLS. Five and ten folds give practically the same curve up to $d = 11$, and both select $d = 11$ (0.0211 and 0.0202). The curve is flat between $d = 8$ and $d = 11$ (0.020 to 0.025), so the degree is not sharply determined. Beyond $d = 11$ the two curves separate: 0.043 against 0.029 at $d = 12$, and 0.053 against 0.025 at $d = 15$. With ten folds each model is trained on 90 points instead of 80. A low-degree fit hardly notices ten extra points; a high-degree fit is sensitive to them, and a single fold with an extreme prediction near the end of the interval can dominate the mean. The choice of K changes how bad the high-degree models look, not which model is selected.

For Ridge we cross-validate over the 15 degrees and

the 23 penalties of Sec. IV B. The lowest CV MSE is at $d = 12$ for both values of K : 0.0209 at $\lambda = 3.2 \times 10^{-5}$ for five folds and 0.0192 at $\lambda = 10^{-5}$ for ten folds (the ten-fold grid is shown in Fig. 16). The region of low error is a broad diagonal valley: the higher the degree, the larger the penalty that is needed. The best Ridge values are only marginally below the best OLS values, by less than the standard error of the estimates (Sec. IV I). What Ridge changes is the behaviour at high degree: at $d = 15$ its best CV MSE is 0.022 and 0.020 for five and ten folds, against 0.053 and 0.025 for OLS.

Do the bootstrap and cross-validation point to the same model complexity? They do not [Table II and Fig. 9(c)]. At low degree the two estimates are similar, with the bootstrap somewhat higher (0.039 against 0.034 at $d = 4$, 0.037 against 0.027 at $d = 6$), and both reject the low degrees. At high degree they diverge: the bootstrap error is 0.18 at $d = 10$ and 106 at $d = 15$, while the CV error stays between 0.02 and 0.06. The bootstrap selects $d = 6$, cross-validation $d = 11$.

The reason is the size of the effective training set. A bootstrap resample of 80 points contains on average about 51 distinct points, so the bootstrap estimates the error of a model trained on a much smaller sample. In this respect it behaves like cross-validation with fewer than three folds [3], whereas five and ten folds train on 80 and 90 distinct points. The difference is largest where the fit is most sensitive to the number and the spacing of the points, that is at high degree. In addition, our bootstrap error rests on one test set of 20 points, whereas cross-validation tests each of the 100 points once. For choosing the degree we therefore trust cross-validation. The bootstrap answers a different question: it splits the error into bias and variance, which cross-validation cannot do, and thereby shows why the high degrees fail.

Table III. Plain gradient descent at η^* for the degrees of Secs. IV A and IV B: condition number κ of the Hessian, iterations needed to reach the closed form to 10^{-6} (at most 10^5), and, where this was not reached, the final distance (in parentheses). Ridge uses $\bar{\lambda} = 10^{-2}$ and converges at every degree.

d	κ	OLS	Ridge	
		Iterations	κ	Iterations
1	1.0	1	1.0	1
2	1.1	5	1.1	5
3	17	92	15.5	84
4	55	376	43.1	290
5	491	3057	178	1019
6	2.1×10^3	15253	269	1718
7	1.6×10^4	$> 10^5$ (3.5×10^{-6})	375	2147
8	7.8×10^4	$> 10^5$ (0.90)	434	2466
10	2.5×10^6	$> 10^5$ (28)	573	3329
12	1.3×10^8	$> 10^5$ (190)	716	4197
15	4.2×10^{10}	$> 10^5$ (670)	939	5474

E. Gradient descent

We now replace the closed forms by gradient descent on the 80 standardised training points. Unless stated otherwise the degree is 5, the degree of the starter code of the project description, and Ridge uses $\bar{\lambda} = 10^{-2}$ ($\lambda = 0.8$). The eigenvalues of the Hessian then range from $\lambda_{\min} = 0.0113$ to $\lambda_{\max} = 5.562$ for OLS ($\kappa = 491$) and from 0.0313 to 5.582 for Ridge ($\kappa = 178$). Standardising pays off here as well: for the unscaled design matrix of the starter code $\kappa = 2050$, four times larger.

The analytical gradients agree with those of JAX to within 2×10^{-16} , the machine precision, for both costs. Used in Eq. (16), the two gradients give the same number of iterations and final parameters that differ by less than 10^{-15} . At the optimal rate η^* (0.3588 for OLS and 0.3563 for Ridge) gradient descent reaches the closed-form solutions to 10^{-6} after 3057 and 1019 iterations. The contraction factor $(\kappa - 1)/(\kappa + 1)$ predicts about 3400 and 1200 iterations for a reduction of the error by 10^6 , and the ratio of the measured counts, 3.0, is close to the ratio of the condition numbers, 2.8. Gradient descent thus converges to the closed-form solution, but far more slowly than the single SVD that the closed form needs.

Figure 11 shows the role of the learning rate. With $0.1 \eta_{\max}$ the distance is still 7×10^{-5} after 2×10^4 iterations; $0.5 \eta_{\max}$ needs 6106 iterations, $0.9 \eta_{\max}$ 3390 and η^* 3057; with $1.02 \eta_{\max}$ the iteration diverges. On the logarithmic scale the curves are straight lines whose slope is set by the flattest direction of the cost, after a fast initial drop in which the steep directions are eliminated. The right panel confirms the bound of Eq. (17): on a grid of 60 learning rates the largest value that converges is 0.3594 for OLS and 0.3581 for Ridge, against the predicted 0.3596 and 0.3583. The number of iterations falls

Table IV. Iterations needed to reach the closed-form solution to 10^{-6} at degree 5, each method at its best learning rate η on a grid of 16 values between 10^{-3} and 1, and the number N of grid values for which the accuracy was reached. RMSprop never reached it; its distance stalls at 1.1×10^{-3} . The last line gives plain gradient descent at η^* , which is not on the grid.

Method	OLS			Ridge, $\bar{\lambda} = 10^{-2}$		
	η	Iter.	N	η	Iter.	N
Plain	0.251	4369	4	0.251	1448	6
Momentum	0.631	243	11	0.251	227	13
AdaGrad	0.0398	5614	10	0.0631	1898	10
RMSprop		never	0		never	0
Adam	0.0158	465	16	0.0631	232	16
Plain at η^*	0.359	3057		0.356	1019	

as η grows, is smallest just below the bound, and rises sharply at the bound, where the factor $1 - \eta \lambda_{\max}$ of the steepest direction approaches -1 . Because κ is large, η^* lies only 0.2% below the bound: the fastest learning rate is next to divergence. The penalty shifts all eigenvalues up by $2\bar{\lambda} = 0.02$, which hardly changes $\lambda_{\max}$ and the bound but nearly triples $\lambda_{\min}$, so Ridge needs about three times fewer iterations at every learning rate. The lecture notes show the same dependence of the iteration count on the learning rate for a model quadratic cost [3].

Table III extends the comparison to the other degrees. The condition number of the OLS Hessian grows from 1 at $d = 1$ to 2×10^3 at $d = 6$ and 4×10^{10} at $d = 15$, and the iteration count grows in proportion to it (between 5 and 8 times κ for $d = 3$ to 6, of the order of the $\ln(10^6)/2 \approx 7$ expected from the contraction factor). Within 10^5 iterations plain gradient descent reaches the OLS solution only up to $d = 6$. At $d = 7$ it is close; from $d = 8$ on it stays far away. The directions it cannot reach are the weak singular modes of Fig. 7, along which the OLS coefficients are largest. With the penalty the condition number stays below 1000 at every degree, and gradient descent converges for all degrees in at most 5474 iterations. Ridge therefore improves the conditioning as well as the variance. For OLS at high degree only the SVD solution is usable.

F. Momentum and adaptive learning rates

Figure 12 and Table IV compare the five update rules on the same two problems, for 16 learning rates between 10^{-3} and 1 and at most 2×10^4 iterations.

Plain gradient descent works only in a narrow window of learning rates, bounded above by $2/\lambda_{\max}$ and below by

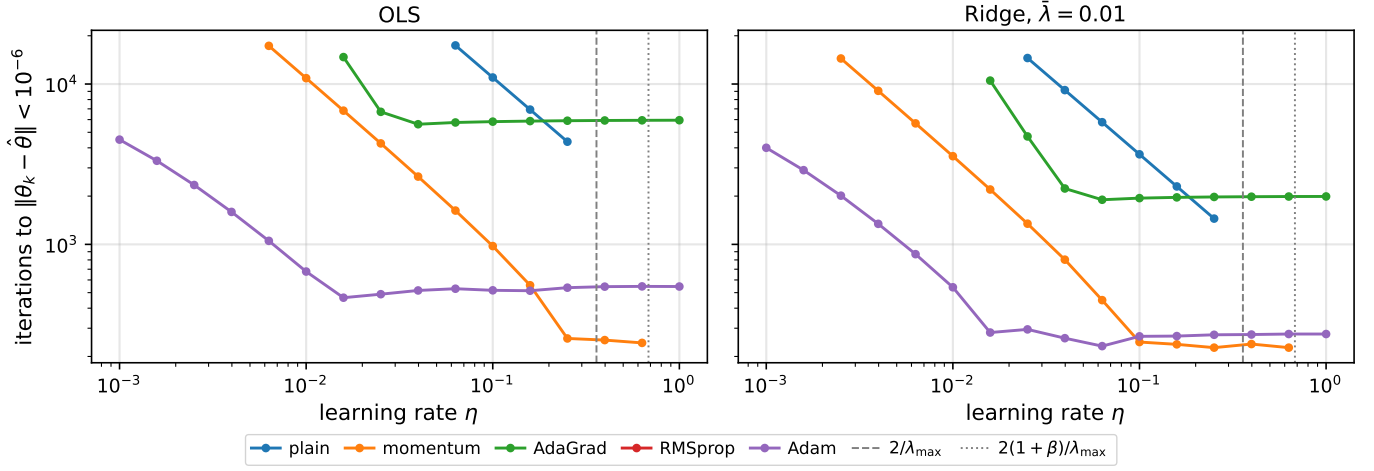


Figure 12. Iterations needed to reach $\|\theta_k - \hat{\theta}\|_2 < 10^{-6}$ against the learning rate for the five update rules at degree 5; OLS (left) and Ridge with $\bar{\lambda} = 10^{-2}$ (right). A missing point means that the accuracy was not reached within 2×10^4 iterations or that the iteration diverged; RMSprop has no points. The vertical lines are the stability bounds of plain gradient descent, $2/\lambda_{\max}$, and of momentum, $2(1+\beta)/\lambda_{\max}$.

slowness: it reaches the accuracy for 4 of the 16 values (OLS). Its best value on the grid, 4369 iterations at $\eta = 0.251$, is higher than the 3057 iterations at η^* , because the grid does not contain $\eta^* = 0.359$ and its next value, 0.398, already lies beyond the bound.

Momentum is the fastest method: 243 iterations for OLS, more than 12 times fewer than plain gradient descent at η^* . This is the expected order of magnitude, since with well-chosen η and β the iteration count scales with $\sqrt{\kappa}$ instead of κ [3, 7], and $\sqrt{491} \approx 22$. In Fig. 12 the momentum curve runs parallel to that of plain gradient descent, shifted to learning rates ten times smaller, as expected for a velocity that builds up to $\eta/(1-\beta) = 10\eta$ times the gradient along a flat direction. Momentum converges up to $\eta = 0.63$, beyond the bound of plain gradient descent but below $2(1+\beta)/\lambda_{\max} = 0.68$, and diverges at $\eta = 1$.

AdaGrad is slower than plain gradient descent at its best (5614 iterations). Its accumulated sum of squared gradients never decreases, so its effective learning rate decays like $1/\sqrt{t}$. It did not diverge for any learning rate on the grid, and for $\eta \geq 0.04$ its iteration count hardly depends on η .

RMSprop never reaches 10^{-6} , for any learning rate and for either cost. Near the minimum the gradient and its running average shrink together, so the update becomes a step of fixed length η that cannot shrink, and the iterate ends in a cycle around the minimum with an amplitude of the order of η . At the smallest learning rate, 10^{-3} , the distance stalls at 1.1×10^{-3} .

Adam reaches the accuracy for all 16 learning rates, in 465 iterations at best. Its step is bounded by about η in every coordinate, so it does not diverge in the sense of Eq. (17), and its averaged gradient $\hat{m}_t$ cancels the sign changes of the gradient near the minimum, which lets it settle where RMSprop cannot. The learning rate

still matters at small values, where the step length limits the speed (4501 iterations at $\eta = 10^{-3}$); for $\eta \geq 0.016$ the count stays near 500. The lecture notes report the same for their regression example: a limit cycle of RMSprop, and convergence of Adam for every learning rate between 10^{-3} and 1 with the best value in the middle of the range [3].

The Ridge penalty helps most the methods that suffer most from the ill-conditioning. Plain gradient descent and AdaGrad need 3.0 times fewer iterations, and Adam 2.0 times fewer. Momentum, which already handles the flat direction by building up speed, hardly changes, and RMSprop is not helped, since its problem is the fixed step length.

G. Lasso regression by subgradient descent

Applied to $|\theta|$, `jax.grad` returns +1 at $\theta = 0$ (and -1 and +1 at $\theta = -0.3$ and 0.3): JAX uses the rule of the branch $\theta \geq 0$. This is a valid subgradient, since +1 lies in the subdifferential $[-1, 1]$; so is the value 0 used in Eq. (8). The two are not equally useful. A fixed choice of +1 pushes a coefficient that sits at zero away from zero in one direction, whereas $\text{sign}(0) = 0$ leaves it there. We therefore use Eq. (8) with the update rules of Sec. IIIH, for $\bar{\lambda} = 10^{-2}$ and degree 5. The reference is the coordinate-descent solution of Scikit-Learn with $\alpha = \bar{\lambda}/2$ and a tolerance of 10^{-12} .

No method reaches a distance of 10^{-6} from the reference, so all runs are continued for 2×10^4 iterations (Fig. 13). The smallest final distances are 5.0×10^{-5} for momentum, 1.0×10^{-4} for plain gradient descent, 2.0×10^{-4} for AdaGrad, 4.0×10^{-4} for Adam and 8.2×10^{-4} for RMSprop; the costs exceed the reference cost by less than 10^{-5} . The reason is the kink of the

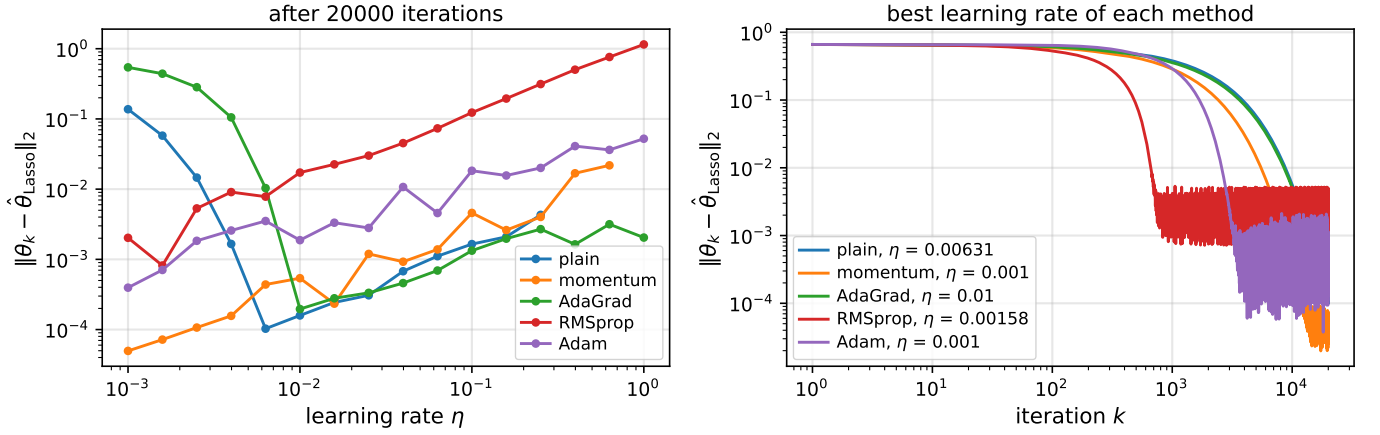


Figure 13. Subgradient descent for the Lasso ($\bar{\lambda} = 10^{-2}$, degree 5) with the five update rules. Left: distance to the Scikit-Learn reference solution after 2×10^4 iterations against the learning rate (a missing point means divergence). Right: distance against the iteration, each method at the learning rate with the smallest final distance.

Table V. Parameters of the standardised columns $x, \dots, x^5$, test MSE and number of parameters that are exactly zero for the three models at degree 5 (80 training and 20 test points of the base case). Ridge and Lasso use $\bar{\lambda} = 10^{-2}$ in the normalised costs of Eqs. (14) and (7). The last line is our subgradient solution with momentum after 2×10^4 iterations.

Model	θ_1	θ_2	θ_3	θ_4	θ_5	Test MSE	Exact zeros
OLS (closed form)	-0.0658	-0.6952	0.2279	0.4944	-0.1412	0.0366	0
Ridge (closed form)	-0.0202	-0.5663	0.0909	0.3703	-0.0535	0.0393	0
Lasso (Scikit-Learn, coordinate descent)	0.0000	-0.5544	0.0158	0.3602	0.0000	0.0394	2
Lasso (own subgradient descent)	0.0000	-0.5544	0.0158	0.3602	0.0000	0.0394	0

penalty. For a coefficient whose optimum is zero, the subgradient contributes $+\bar{\lambda}$ when the coefficient is slightly positive and $-\bar{\lambda}$ when it is slightly negative, so every step pushes it across zero by about $\eta\bar{\lambda}$. The gradient does not vanish at the solution, as it does for the smooth costs, and the distance levels off at a value set by the learning rate. For plain gradient descent $\eta\bar{\lambda} = 6 \times 10^{-5}$ at its best learning rate, against a final distance of 1.0×10^{-4} .

The learning rate therefore trades speed against accuracy (left panel of Fig. 13). At small η a method has not yet arrived after 2×10^4 steps, and a larger η helps; once it has arrived it oscillates by about $\eta\bar{\lambda}$, and a larger η hurts. Plain gradient descent and AdaGrad show both branches, with minima at $\eta = 0.0063$ and 0.01 . Momentum, Adam and RMSprop arrive even at the smallest learning rates of the grid, so mainly the rising branch is visible. In the right panel RMSprop arrives first (distance below 10^{-3} after 832 iterations) but then oscillates in a band between about 10^{-3} and 5×10^{-3} ; Adam follows after 3241 iterations and oscillates between about 10^{-4} and 2×10^{-3} ; momentum settles lowest. Unlike for the smooth costs, Adam no longer settles: the subgradient keeps switching between $+\bar{\lambda}$ and $-\bar{\lambda}$ near zero, so the averaging that damps its steps near a smooth minimum does not remove them.

Table V compares the three models at degree 5. OLS uses all five powers. Ridge shrinks all of them and sets

none to zero. The Lasso sets the coefficients of x and x^5 exactly to zero, shrinks that of x^3 to 0.016 and keeps x^2 and x^4 : it selects the even powers, as the symmetry of f suggests. Our subgradient solution agrees with the reference to four decimals in every coefficient and in the test MSE, but it has no exact zeros (the coefficients of x and x^5 end at 4×10^{-5} and 2×10^{-5}). The test MSE is 0.0366 for OLS and 0.0393 and 0.0394 for Ridge and Lasso: at degree 5 there is little variance to remove, and the penalties mainly add bias. Subgradient descent with a fixed learning rate is thus a poor Lasso solver. Its accuracy is limited by the learning rate, and it does not produce the exact zeros that are the purpose of the Lasso; coordinate descent with soft thresholding does both.

H. Stochastic gradient descent

We now use minibatches of $M = 1, 5, 20$ and 80 points; $M = 80$ is the full batch, that is, no SGD. Each update rule uses one constant learning rate for all M (Table VI), chosen so that $M = 1$ does not diverge: the cost of a single data point has a curvature of up to $2\|\mathbf{x}_i\|^2 = 92$, so plain SGD needs $\eta < 2/92 = 0.022$.

Figure 14 and Table VI show the excess cost for OLS over 1000 epochs. The batch size is a tradeoff. A small batch makes many updates per epoch and progresses

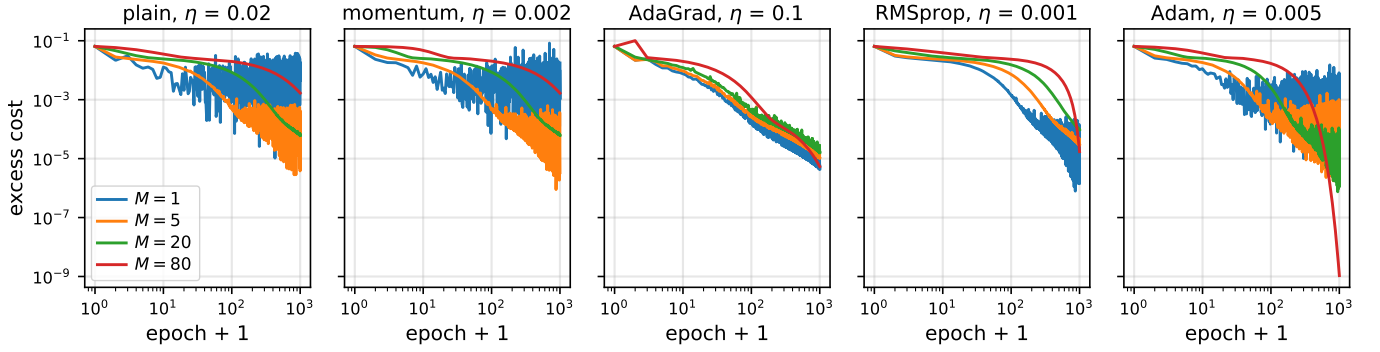


Figure 14. Excess cost $C(\theta) - C(\hat{\theta}_{\text{OLS}})$ of minibatch SGD against the epoch for the batch sizes $M = 1, 5, 20$ and 80 (full batch) and the five update rules, each with one constant learning rate η (OLS, degree 5, 1000 epochs, 80 training points).

Table VI. Excess cost of SGD for OLS after 1000 epochs (median over the last 100 epochs), in units of 10^{-5} , for the runs of Fig. 14. $M = 80$ is the full batch.

Method	η	$M = 1$	$M = 5$	$M = 20$	$M = 80$
Plain	0.02	380	12	6.9	190
Momentum	0.002	350	4.3	6.9	190
AdaGrad	0.1	0.71	1.4	2.1	0.66
RMSprop	0.001	3.2	3.6	11	5.3
Adam	0.005	110	9.5	1.1	0.00045

Table VII. With and without SGD after 500 epochs: excess cost (median over the last 100 epochs) in units of 10^{-5} . Full batch: 500 steps at the learning rate of Table IV; SGD: $M = 5$ at the learning rate of Table VI. Entries marked ≈ 0 are below 10^{-13} in absolute value.

Method	OLS		Ridge, $\bar{\lambda} = 10^{-2}$	
	Full batch	SGD	Full batch	SGD
Plain	2.9	13	0.011	9.8
Momentum	≈ 0	6.9	≈ 0	3.8
AdaGrad	42	4.4	0.091	0.73
RMSprop	770	15	410	1.4
Adam	≈ 0	11	≈ 0	11

fastest at first: after 10 epochs $M = 1$ is ahead for every method. But its gradient noise does not vanish near the minimum, where the full gradient goes to zero while the gradients of single points do not. With a constant learning rate the iterate therefore ends on a noisy plateau, at an excess cost of about 4×10^{-3} for plain SGD and momentum and 10^{-3} for Adam at $M = 1$. A larger batch lowers the plateau but makes fewer updates per epoch. For plain SGD and momentum an intermediate batch ($M = 5$ or 20) is best after 1000 epochs, while the full batch, restricted to a learning rate that was chosen for $M = 1$, is still far from converged. For Adam the plateau falls with the batch size, and the full batch converges to 4.5×10^{-9} . AdaGrad depends least on M : its built-in decay of the step acts like a schedule that also reduces the noise. RMSprop is slow for every batch size, because its steps have a length of about $\eta = 10^{-3}$. More epochs do not lower the plateaus of the SGD runs with a constant learning rate; only the full-batch runs, which have no noise, keep descending.

A decaying learning rate removes the plateau (Fig. 15, plain SGD with $M = 5$). A constant $\eta = 0.05$ is fast at first but stays in a noisy band around 1.2×10^{-3} ; a constant $\eta = 0.01$ is slower but reaches 1.6×10^{-5} after 2000 epochs. The four schedules of Eq. (22) all start at $\eta_0 = 0.05$ and differ in the number of updates t_1 after which they decay (there are 16 updates per epoch). The

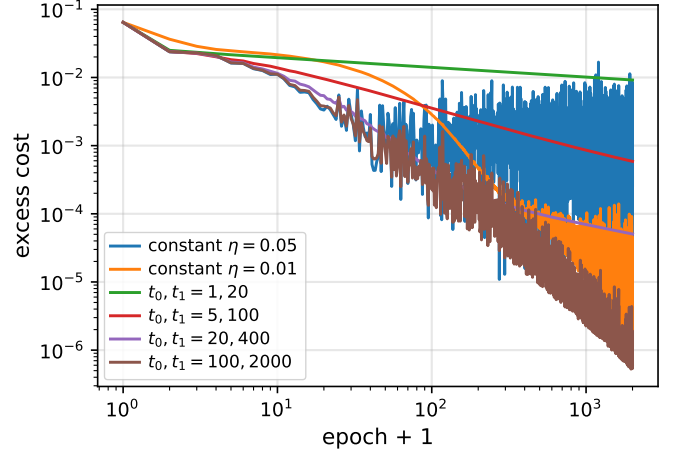


Figure 15. Plain SGD with $M = 5$ (OLS, degree 5): excess cost against the epoch for two constant learning rates and for four schedules $\eta_t = t_0/(t + t_1)$, Eq. (22), which all start at $\eta_0 = 0.05$.

decay time decides the outcome: the final excess cost is 9.2×10^{-3} , 6.0×10^{-4} , 5.1×10^{-5} and 9.5×10^{-7} for $t_1 = 20, 100, 400$ and 2000 . With $t_1 = 20$ the learning rate decays after about one epoch, and the iterate freezes far from the minimum. Neglecting the noise, the error

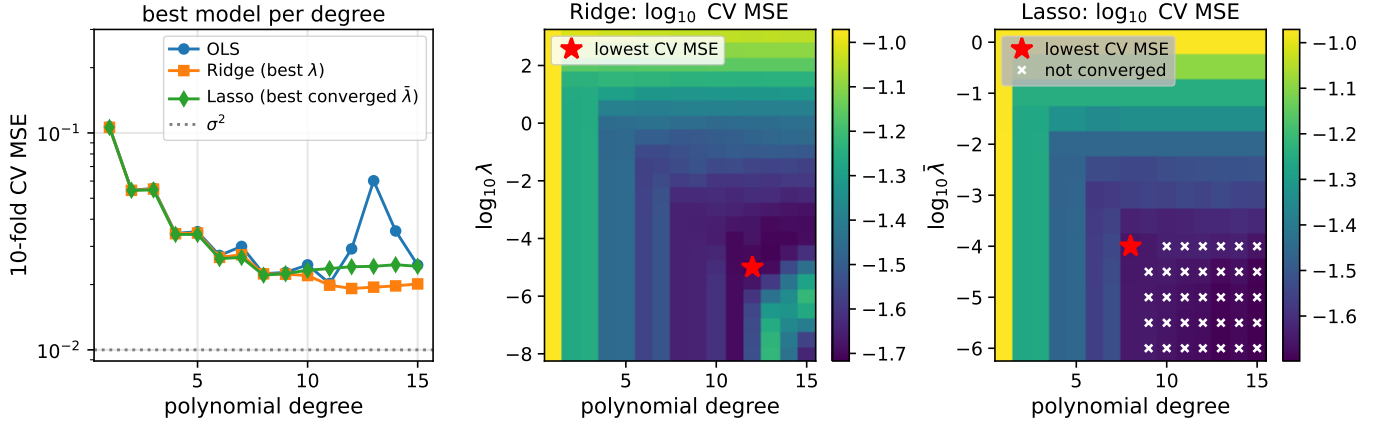


Figure 16. Final model selection with 10-fold cross-validation ($n = 100$, $\sigma = 0.1$). Left: CV MSE against the degree for OLS, and for Ridge and Lasso at the best penalty for each degree. Middle and right: $\log_{10}$ of the CV MSE on the grid of degree and penalty, for Ridge [λ of Eq. (3)] and for the Lasso [$\bar{\lambda}$ of Eq. (7)]. Stars mark the lowest CV MSE. White crosses mark Lasso fits that did not converge; they are excluded from the selection.

Table VIII. Model selected for each method by 10-fold cross-validation ($n = 100$, $\sigma = 0.1$), with the standard error (SE) of the CV estimate. The penalty is λ for Ridge and $\bar{\lambda}$ for the Lasso (converged fits only).

Method	Degree	Penalty	CV MSE	SE
OLS	11		0.0202	0.0031
Ridge	12	10^{-5}	0.0192	0.0026
Lasso	8	10^{-4}	0.0221	0.0021

Table IX. Bootstrap error, measured bias² and variance, Eq. (13), of the three methods at two degrees, with the split and the resamples of Fig. 9(a) ($B = 100$). Ridge and Lasso use the penalty with the lowest CV MSE at that degree.

d	Method	Penalty	Error	Bias ²	Variance
5	OLS		0.0406	0.0362	0.0044
5	Ridge	0.32	0.0396	0.0371	0.0026
5	Lasso	3.2×10^{-3}	0.0387	0.0361	0.0026
12	OLS		0.7897	0.0612	0.7285
12	Ridge	10^{-5}	0.1556	0.0400	0.1156
12	Lasso	3.2×10^{-4}	0.0412	0.0262	0.0150

along an eigendirection of curvature λ_i is multiplied by $\prod_t (1 - \eta_t \lambda_i) \approx [(T + t_1)/t_1]^{-t_0 \lambda_i}$ after T updates, a power law instead of the exponential decay at constant rate. For the flattest direction and the $T = 32000$ updates of our runs this factor is 0.92, 0.72, 0.37 and 0.04 for the four schedules. The conditions of Robbins and Monro [11], $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$, express two needs: the steps must not shrink so fast that the minimum cannot be reached, and they must shrink eventually so that the noise dies out. Every schedule of the form of Eq. (22) satisfies them for an infinite run. In a finite run the constants decide. With $t_1 = 20$ the sum of the steps is too short in practice, whereas the last schedule, which keeps η_0 for about 125 epochs before it decays, meets both needs and gives the lowest excess cost of all runs.

Table VII compares every method with and without SGD after 500 epochs, which is the same arithmetic in both cases. Full-batch momentum and Adam reach the closed-form solutions to rounding accuracy. Plain gradient descent has not converged after 500 steps, and Ada-Grad and RMSprop are slower still, at learning rates that were selected for runs of 2×10^4 iterations. With SGD every method stops at its noise level, an excess cost between

4×10^{-5} and 1.5×10^{-4} for OLS and between 7×10^{-6} and 1.1×10^{-4} for Ridge. SGD reaches the closed forms only up to this level; it is better than the full batch only where the full-batch learning rate was a poor choice for a short run.

The measured time per epoch on our computer was 0.36 ms for $M = 1$, 0.081 ms for $M = 5$, 0.027 ms for $M = 20$ and 0.014 ms for the full batch. Although an epoch costs the same arithmetic for every M , SGD with single points is about 25 times slower in practice, because the overhead of each update in the Python loop exceeds the arithmetic of a gradient over a few points. For $n = 80$ and five parameters SGD therefore brings no advantage: a full gradient is cheap, full-batch momentum or Adam reach the closed form in a few hundred iterations, and SGD needs a tuned learning rate or schedule and still stops at a noise level. Its advantage appears when n is large, where one epoch of SGD makes n/M useful updates for the cost of a single full gradient.

I. Final model selection

We finally select a model for each of the three methods by 10-fold cross-validation on all 100 points, with the pipelines of Sec. III F. OLS is scanned over the degrees 1 to 15, Ridge over the same degrees and the 23 penalties λ between 10^{-8} and 10^3 , and the Lasso over the degrees and 13 penalties $\bar{\lambda}$ between 10^{-6} and 1. OLS and Ridge are solved exactly; the Lasso is solved by the coordinate descent of Scikit-Learn with at most 10^5 iterations. We record whether the Lasso solver converged in every fold. It did not in 34 of the 195 cells, all at degrees 9 to 15 and $\bar{\lambda} \leq 10^{-4}$, where the nearly collinear columns slow the solver down as they slowed gradient descent in Table III. An unconverged fit is not the Lasso solution, since stopping early acts as an additional, uncontrolled regularisation. Such cells are marked in Fig. 16 and excluded from the selection. The check matters: the lowest CV MSE of the whole Lasso grid (0.0200, at $d = 15$ and $\bar{\lambda} = 3.2 \times 10^{-6}$) belongs to an unconverged cell.

Table VIII lists the selected models. Ridge at $d = 12$ and $\lambda = 10^{-5}$ has the lowest CV MSE, followed by OLS at $d = 11$ and the Lasso at $d = 8$. The standard errors of these estimates are 0.002 to 0.003, so the three methods differ by about one standard error or less: the data do not single out one of them. All three errors are about twice $\sigma^2 = 0.01$. Part of this excess is specific to our sample, whose realised noise is larger than its expectation (variance 0.0150 in the training points, Sec. IV A); the rest is the bias and the variance of the fitted polynomials.

The left panel of Fig. 16 shows where the methods differ. Up to $d = 9$ the three curves coincide, because at low degree there is little variance and a penalty has nothing to remove. Beyond that the OLS error rises irregularly (0.060 at $d = 13$), while the Ridge and Lasso errors stay flat, at 0.019 to 0.020 and at 0.023 to 0.025. The value of the penalties for this problem is not a lower error but a result that no longer depends on the degree. The Lasso curve lies above the Ridge curve at high degree. Since its fits with small penalties did not converge there and were excluded, we cannot tell whether this is a property of the Lasso or of the solver.

Which term of Eq. (11) does each method act on? OLS has no penalty: it lowers the bias by raising the degree and pays with variance. Ridge and Lasso reduce the variance at the price of some bias, Ridge by shrinking the weakly determined singular directions and the Lasso by shrinking and removing coefficients. To test this reading we repeated the bootstrap of Sec. IV C with the three pipelines at $d = 5$ and $d = 12$ (Table IX). At $d = 12$, where OLS is dominated by variance, Ridge reduces the variance by a factor of 6 and the Lasso by a factor of 50, and the error falls from 0.79 to 0.16 and 0.04. At $d = 5$, where the variance is already small, the penalties lower it only from 0.0044 to 0.0026, the measured bias stays the same or rises slightly, and the error hardly changes. The numbers support the reading that the penalties act on the variance. The comparison of the biases at $d = 12$

is less clean, because the measured bias of OLS is itself inflated there (Sec. IV C).

In summary, OLS has an exact closed form and a single hyperparameter, the degree, but at high degree it is unstable, so the degree must be chosen with care. Ridge has a closed form for every λ , is stable at any degree because it removes the weakest singular directions, improves the conditioning for gradient descent, and has the lowest CV error here; it has two hyperparameters and, since it sets no coefficient to zero, does not show which powers matter. The Lasso selects powers and reduces the variance strongly, but it has no closed form, and both our subgradient method (no exact zeros) and coordinate descent (no convergence at high degree) struggle with the collinear columns, so its optimum at high degree remains uncertain.

V. CONCLUSION

We fitted Runge’s function with polynomials of degree up to 15 to 100 noisy points. The error of OLS on new data is lowest at an intermediate degree, 11 according to cross-validation, with a shallow minimum of about 0.02. The bootstrap shows why. The bias falls with the degree, while the variance grows by almost four orders of magnitude between degree 6 and degree 15, because the monomials are nearly collinear and the fit amplifies the noise along the weak singular directions of the design matrix. The bootstrap itself selects degree 6, since its resamples contain only 63% of the distinct points, which penalises the flexible models. Ridge removes the weak directions one after another as the penalty grows, and the Lasso sets coefficients exactly to zero. Both reduce the variance and make the cross-validated error flat in the degree, but their best errors (0.019 and 0.022) are within about one standard error of the best OLS error (0.020). Gradient descent reproduces the closed-form solutions as long as the condition number allows it: at the optimal learning rate it needs 3057 iterations at degree 5, it fails beyond degree 6 for OLS, and it converges at every degree with a small Ridge penalty. Momentum is more than 12 times faster, Adam is the least sensitive to the learning rate, and RMSprop with a constant learning rate does not converge.

Several limitations qualify these results. All data come from one seed. The curves of Secs. IV A and IV B rest on a single split with 20 test points, and where a degree or a penalty was selected on that test set the quoted error is optimistic. We did not repeat the analysis over many seeds, so the bootstrap curves have no error bars, and the standard errors of the cross-validation (0.002 to 0.003) are as large as the differences between the three methods: our ranking of OLS, Ridge and Lasso is not statistically resolved. The bootstrap estimate of the bias is contaminated at high degree by the finite number of resamples, and the bootstrap error itself is pessimistic there. For the Lasso our own code is a subgradient method that

Table X. Level of LLM contribution to the text of each section (0 none, 1 language only, 2 editorial, 3 generative).

Section	Level	Notes
Abstract	3	drafted from the results of the notebook
Introduction	3	drafted along the course guide on report writing
Methods	3	drafted from the theory cells of the notebook and the lecture notes
Implementation and tests	3	drafted from the code and its printed checks
Results and discussion	3	drafted from the outputs and the observation cells of the notebook
Conclusion	3	drafted from the results and the caveats noted in the notebook
This appendix	3	drafted from the LLM notes in the notebook

produces no exact zeros, and in the final selection 34 cells of the grid had to be excluded because the library solver did not converge, so the optimum of the Lasso at high degree is unknown. The optimisers were compared at one degree, on one grid of 16 learning rates and with standard values of their other hyperparameters; the best learning rate of plain gradient descent on that grid needs 43% more iterations than the optimal one, which shows how much the grid matters. Convergence was measured against the closed-form solution, which is not available in practice, and the learning rates for SGD were chosen for stability at $M = 1$, not tuned for each batch size. The computing times depend on the machine. Finally, we relied on Scikit-Learn for cross-validation and did not write our own K -fold code.

Two extensions follow directly from these limitations. The ill-conditioning that limits gradient descent, coordinate descent and the bootstrap is a property of the monomial basis, not of the problem. Replacing the powers x^j by orthogonal polynomials, such as the Legendre or Chebyshev polynomials, should reduce the condition number of the design matrix by many orders of magnitude and make the iterative solvers usable at high degree. For the Lasso, a proximal gradient method, in which every gradient step on the squared error is followed by soft thresholding, would give exact zeros with our own code and allow a fair comparison with coordinate descent. With these two changes the analysis could be repeated over many independent data sets, which is possible here because f is known, in order to obtain the true bias and variance and to measure how far the bootstrap estimates are from them.

Beyond the results, the project changed how we would work next time. The most practical lesson concerned

Table XI. Level of LLM contribution to the code (0 none, 1 debugging, 2 snippet, 3 skeleton, 4 substantial).

File	Level	Description
Project1.ipynb , parts a) to i)	4	Claude wrote the code cells of all parts, starting from the course notebooks of weeks 35 to 38 and the starter cells of the project description, as stated above every cell. The functions <code>runge</code> , <code>MSE</code> , <code>R2</code> , <code>ridge_svd</code> , <code>optimiser_step</code> , <code>make_batches</code> and <code>step_length</code> are course code to which comments were added.
export_figures.py	4	Script written by Claude that executes the notebook unchanged and saves its figures as PDF at the size of the report.

conventions. The same penalty takes different numerical values in the closed-form Ridge solution, in the cost normalised by $1/n$ that gradient descent minimises, and in the Lasso of Scikit-Learn, and only the comparison with a closed-form solution made these factors visible. We also learned not to trust a single test split: for the same data the single split, the bootstrap and cross-validation selected degrees 8, 6 and 11. A library solver needs the same checks as our own code, since the lowest Lasso error of the whole grid came from fits that had not converged. We would now write the tests before the analysis and plan repeated runs over several seeds from the start.

Appendix A: Use of AI/LLM tools

This declaration follows the guidelines of the course [19].

A.1 Tools used

Claude (Claude Opus 5.5, accessed through claude.ai, September and October 2026).

A.2 Text writing and editing

Table X gives the contribution level for every section. All sections are at Level 3 (generative).

Prompt and process (Level 3). We gave Claude our notebook with all its outputs, the project description and the course material on report writing (grading form, writing guide, L^AT_EX template and the LLM guidelines). We

asked it to write this report in L^AT_EX according to those criteria and to check every paragraph against the other sections, the course material and the results. The theory cells and the observation cells of the notebook, which the Methods and Results sections follow, had themselves been drafted with Claude in September, from the outputs of our code and from our follow-up questions. In the same session Claude executed the notebook again, exported its figures as PDF at the size of this report, and compared the numbers in the text with the printed outputs. The numerical results in this report are outputs of the notebook. A few quantities derived from them were computed by Claude: the ratios quoted in the text, the iteration counts expected from the condition number (Sec. IV E), the decay factors of the schedules (Sec. IV H) and the estimate of the resampling contribution to the measured bias (Sec. IV C).

A.3 Code generation and assistance

Table XI gives the contribution level for every code file. *Prompt and verification (Level 4)*. For each part we gave Claude the project text and the course notebook of the corresponding week and asked it to adapt the course code to our problem and to explain every line. We ran every cell ourselves and verified the results against NumPy, Scikit-Learn and the closed-form solutions (Table I). In the notebook every function to which the LLM contributed carries an LLM-assisted tag in its docstring, and every code cell is preceded by a markdown note that states its source in the course material and the level of assistance.

Our changes and checks. We went through the text and the code, asked Claude for further explanations wherever something was unclear, and cross-checked the content against the lecture material. These follow-up questions shaped both the text and the code.

-
- [1] C. Runge, Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten, *Z. Math. Phys.* **46**, 224 (1901).
 - [2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. (Springer, New York, 2009).
 - [3] M. Hjorth-Jensen, Machine learning and the physical sciences: From discriminative methods to generative methods and reinforcement learning, <https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/BookML/Lecturebook.pdf> (2026), lecture notes for FYS-STK3155/4155, University of Oslo, version of September 23, 2026.
 - [4] A. E. Hoerl and R. W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, *Technometrics* **12**, 55 (1970).
 - [5] R. Tibshirani, Regression shrinkage and selection via the lasso, *J. R. Stat. Soc. B* **58**, 267 (1996).
 - [6] B. Efron, Bootstrap methods: Another look at the jackknife, *Ann. Stat.* **7**, 1 (1979).
 - [7] B. T. Polyak, Some methods of speeding up the convergence of iteration methods, *USSR Comput. Math. Math. Phys.* **4**, 1 (1964).
 - [8] J. Duchi, E. Hazan, and Y. Singer, Adaptive subgradient methods for online learning and stochastic optimization, *J. Mach. Learn. Res.* **12**, 2121 (2011).
 - [9] T. Tieleman and G. Hinton, Lecture 6.5, RMSProp: Divide the gradient by a running average of its recent magnitude, COURSE: Neural Networks for Machine Learning (2012).
 - [10] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, in *International Conference on Learning Representations (ICLR)* (2015) [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).
 - [11] H. Robbins and S. Monro, A stochastic approximation method, *Ann. Math. Stat.* **22**, 400 (1951).
 - [12] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, Automatic differentiation in machine learning: a survey, *J. Mach. Learn. Res.* **18**, 1 (2018).
 - [13] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, JAX: composable transformations of Python+NumPy programs, <http://github.com/google/jax> (2018).
 - [14] M. Hjorth-Jensen, FYS-STK3155/4155 Applied Data Analysis and Machine Learning: lecture slides, notebooks and exercises of weeks 34 to 38, <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/WeeklyMaterial> (2026), University of Oslo.
 - [15] Department of Physics, University of Oslo, Project 1 on machine learning: Regression analysis, resampling methods and gradient descent, <https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc/Projects/2026/Project1> (2026), project description and starter notebook, FYS-STK3155/4155.
 - [16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, Scikit-learn: Machine learning in Python, *J. Mach. Learn. Res.* **12**, 2825 (2011).
 - [17] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, Array programming with NumPy, *Nature* **585**, 357 (2020).
 - [18] J. D. Hunter, Matplotlib: A 2D graphics environment, *Comput. Sci. Eng.* **9**, 90 (2007).
 - [19] Department of Physics, University of Oslo, Guidelines for declaring the use of large language models in project reports, https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md (2026), FYS-STK3155/4155.